1 Definitions

Reinforcement learning: learning paradigm where agent interacts at dicrete time steps $t = 0, 1, \ldots, k$. At step t, the agent observes $s_t \in \mathcal{S}$, acts with $a_t \in \mathcal{A}$, receives reward $r_{t+1} \in \mathbb{R}$, and transitions to $s_{t+1} \in \mathcal{S}$.

- Optionally, agent can receive $o_t \in \mathcal{O}$; but the state can be modelled as a sequence of observations.

Markov Decision Process: A tuple $M = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$ corresponding to the state space, action space, environment transition $P(s_{t+1} \mid s_t, a_t)$, reward function $r(s_t, a_t)$, discount factor γ.

- Implies no need for history of states.

Policy: A distribution over actions conditioned on states $\pi(a \mid s)$ for all $a \in \mathcal{A}$.

- The policy is **stationary** in that the mapping from states to actions does not change.

Rewards: Provided by the environment to encourage goal-seeking behavior: $r(s, a) = \mathbf{E}\left[R_{t+1} \mid S_t, A_t\right]$.

Return: Reward aggregate over time. Defined as

$$G_t = R_{t+1} + \cdots + R_T$$

for **episodic tasks**, or

$$G_t = \sum_{k=0} \gamma^t R_{t+k+1}$$

for **continuing tasks**.

Optimal policy: The goal of RL is to determine

$$\pi^* := \underset{\pi \in \Pi}{\operatorname{argmax}}\, \mathbf{E}_{\tau \sim P_\pi(\cdot)} \left[\sum_{t=0}^{T} r(s_t, a_t) \right]$$

Occupancy Distribution: The amount of time an agent spends in a (state, action) pair:

$$d_\pi(s, a) := \sum_{t=0}^{T-1} P_\pi^t(s, a)$$

where $P_\pi^t(s, a)$ is the probability at time t the agent is at (s, a) when following policy π.

State-value function: The expected return from the state s following π:

$$v_\pi(s) := \mathbf{E}\left[G_t \mid s_t = s\right].$$

Action-value function: The expected return from the state s taking some action a following π:

$$q_\pi(s, a) := \mathbf{E}\left[G_t \mid s_t = s, a_t = a\right].$$

Optimal state/action-value function: Defined as $v^*(s) := v_{\pi^*}(s)$ and $q^*(s,a) := q_{\pi^*}(s,a)$.

Policy performance: Expected return of a policy:

$$J(\pi) = \mathbf{E}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)\right] = \mathbf{E}_{s_0 \sim \rho_0}\left[v_\pi(s_0)\right].$$

Prediction Problem: Given an MDP and policy, determine the state and action value functions.

Optimal Control: Given an MDP, find π^*.

Given v^* for a discrete action-space, we can derive π^* via

$$\pi^*(a \mid s) = \operatorname*{argmax}_{a \in \mathcal{A}} \sum_{s',r} P(s', r \mid s, a)(r + \gamma v^*(s')).$$

2 Value-Based Methods

Three cases for value-based methods: finite with known rewards and dynamics, finite with unknown rewards and dynamics, and infinite.

2.1 Tabular Methods

Bellman Expectation Equations (value form):

$$v_\pi(s) = \sum_a \pi(a \mid s)\left[r(s,a) + \sum_{s'} P(s' \mid s, a) v_\pi(s')\right]$$

Proof:

$$\begin{aligned}
v_\pi(s) &= \mathbf{E}\left[G_t \mid s_t = s\right] \\
&= \mathbf{E}\left[R_{t+1} + \gamma G_{t+1} \mid s_t = s\right] \\
&= \mathbf{E}\left[R_{t+1} \mid s_t = s\right] + \gamma \mathbf{E}\left[G_{t+1} \mid s_t = s\right] \\
&= \sum_a \pi(a \mid s) r(s,a) + \gamma \sum_a \pi(a \mid s) \mathbf{E}\left[G_{t+1} \mid s_t = s, a_t = a\right] \\
&= \sum_a \pi(a \mid s) r(s,a) + \gamma \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) v_\pi(s') \\
&= \sum_a \pi(a \mid s)\left[r(s,a) + \sum_{s'} P(s' \mid s, a) v_\pi(s')\right]
\end{aligned}$$

Bellman Expectation Equations (action form):

$$q_\pi(s) = r(s,a) + \sum_{s'}\left[P(s' \mid s, a) \sum_{a'} \pi(a' \mid s') q_\pi(s', a')\right]$$

Proof:

$$\begin{aligned} q_\pi(s) &= \mathbf{E}\left[G_t \mid s_t = s, a_t = a\right] \\ &= r(s,a) + \gamma \mathbf{E}\left[G_{t+1} \mid s_t = s, a_t = a\right] \\ &= r(s,a) + \sum_{s'} P(s' \mid s,a) \mathbf{E}\left[G_{t+1} \mid s_{t+1} = s'\right] \\ &= r(s,a) + \sum_{s'} \left[P(s' \mid s,a) \sum_{a'} \pi(a' \mid s') q_\pi(s',a') \right] \end{aligned}$$

Relationship between v_π and q_π:

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s,a)$$

Bellman Optimality Equation (value form):

$$v^*(s) = \max_a r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v^*(s')$$

Bellman Optimality Equation (action form):

$$q^*(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} q^*(s',a')$$

Relationship between v^* and q^*:

$$v^*(s) = \max_a q^*(s,a)$$

Solving Bellman Expectation Equations: Value functions are a linear system of equations:

$$\begin{aligned} v_\pi &= r_s^\pi + \gamma T_{s's}^\pi v_\pi \\ v_\pi &\in \mathbb{R}^{|\mathcal{S}| \times 1} \\ r_s^\pi &= \sum_a \pi(a \mid s) r(s,a) \in \mathbb{R}^{|\mathcal{S}| \times 1} \\ T_{s's}^\pi &= \sum_a \pi(a \mid s) P(s' \mid s,a) \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|} \end{aligned}$$

This can be solved using matrix inversion $v_\pi = (I - \gamma T^\pi)^{-1} r^\pi$.

Advantage:

$$A_\pi(s,a) := q_\pi(s,a) - v_\pi(s)$$

Bellman Expectation Operator:

$$(T_\pi v)(s) = \sum_a \pi(a \mid s) \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v(s') \right]$$

Bellman Optimality Operator:

$$(T^*v)(s) = \max_a r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v(s')$$

Iterative Policy Evaluation: Iteratively applies the Bellman expectation operator to v^*. This eventually converges to v^*:

$$v_{[k+1]}(s) := T_\pi v_{[k]}.$$

Value Iteration: Iteratively applies the Bellman optimality operator to v^* until convergence:

$$v_{[k+1]}(s) := T^* v_{[k]}.$$

Policy Iteration: Run policy evaluation until convergence to compute v_{π_k} (e.g. closed form, iterative policy evaluation, etc.). Then, recompute π via

$$\pi_k(a \mid s) = \operatorname*{argmax}_a r(s,a) + \sum_{s'} P(s' \mid s,a) v_{\pi_{k-1}}(s')$$

- Value iteration is a special case of policy iteration with a single policy evaluation at each step. Bellman optimality operators do not require a policy π, so computing this can be skipped.

Limitations of tabular methods: computation required for all states, even ones that are unlikely. Not applicable to continuous state or action spaces.

2.2 Model-Free Prediction

Monte-Carlo Policy Evaluation: Learn $v_\pi(s)$ from episodes of experience under π:

$$N(s_t) \leftarrow N(s_t) + 1$$
$$V(s_t) \leftarrow V(s_t) + \frac{1}{N(s_t)}(G_t - V(s_t))$$

- For non-stationary problems, track running means $V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$.
- Update rule applies to both **first-visit MC** or **every-visit MC**.
- Monte-Carlo control uses the same algorithm for the action value function.

Monte-Carlo Control: An algorithm that alternates between Monte-Carlo policy evaluation and policy improvement. The most basic form of policy improvement is given by

$$\pi(s) \leftarrow \operatorname*{argmax}_a q(s,a).$$

- MC control is **on-policy** because trajectories collected by the policy are used to improve it in the following step and then discarded.

Exploration Problem: Policy needs to act suboptimally to explore all actions.

ϵ-soft Policies: The probability of an action $\pi(a \mid s)$ is $\epsilon / |\mathcal{A}(s)|$ for the non-optimal action or $(1-\epsilon) + \frac{\epsilon}{|\mathcal{A}(s)|}$ otherwise. In Monte-Carlo policy evaluation, this is used to address the exploration problem.

- Disadvantage: The second-best action is ranked equally with the worst-possible action.

Temporal Difference Prediction: **TD(0)** bootstraps MC policy evaluation with current goal estimate after one step:

$$V(s_t) \leftarrow V(s_t) + \alpha(R_{t+1} + \gamma V(s_{t+1}) - V(s_t)).$$

The term $R_{t+1} + \gamma V(s_{t+1})$ is called the **TD target**.

- Compared to MC policy iteration, TD has lower variance and is instantaneous (can be evaluated online, instead of waiting for G_t to be computed after following the whole trajectory).

n-step TD Learning: Uses goal $G_t^{(n)}$ defined as

$$G_t^{(n)} := R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(s_{t+n})$$

- TD(0)$\equiv$ 0-step TD learning, MC control $\equiv \infty$-step TD learning. In general, increasing n leads to higher variance, lower bias.

SARSA: Uses action-valued TD learning:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

For each step, take action a and observe new state s' and reward r. Pick a' from s' from policy derived from Q (e.g. ϵ-greedy). SARSA is on-policy.

Q-Learning [1]: Performs greedy, off-policy TD learning:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(R_{t+1} + \gamma \max_a Q(s', a) - Q(s_t, a_t)).$$

- SARSA is "safe" because it accounts for random steps, e.g. in tasks like cliff-walking. Q-learning is capable of taking risks because it assumes that the optimal policy is used for future steps.
- Reduces variance from SARSA, but introduces maximization bias.

Maximization Bias: Consider (s, a) s.t. $q^*(s, a) = 0$ and $Q(s, a) > 0$. Then, $Q(s, \operatorname{argmax}_a Q(s, a)) > 0$, but $q^*(s, \operatorname{argmax}_a q^*(s, a)) = 0$. This results from using the same Q function to compute and evaluate the argmax.

Double Q-Learning: Train two functions Q_1, Q_2. Perform Q-learning on different time steps and pick at reandom which function to update. Use Q_2 to compute the next state for Q_1 and vice versa:

$$a^* \leftarrow \operatorname*{argmax}_a Q_1(s', a)$$
$$Q_1(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(R_{t+1} + \gamma Q_2(s', a^*) - Q_1(s_t, a_t))$$

This works because it decouples selecting the best action from assigning a value to the best action.

Expected SARSA: Uses an **expected TD target**:

$$r + \gamma \mathbf{E}_{a \sim \pi(\cdot \mid s_{t+1})} [Q(s_{t+1}, a)] = r + \gamma \sum_a \pi(a \mid s_{t+1})) Q(s_{t+1}, a)$$

- When using a greedy policy (not ϵ-greedy), SARSA, Q-learning, and expected SARSA are equivalent.
- Reduces variance from SARSA.

2.3 Deep RL Methods

Function approximation: represent Q-values as a learned function parameterized by θ.

Neural representations of value functions: Approximate v_π and q_π with $V_\theta(s_t)$ and $Q_\theta(s_t, a_t)$ where $|\theta| < |\mathcal{S}|$. Goal is to find

$$\underset{\theta}{\operatorname{argmin}} J(\theta) = \mathbf{E}_\pi \left[(v_\pi(s) - V_\theta(s))^2 \right].$$

The corresponding gradient decent updates are

$$\Delta\theta = -\frac{1}{2}\alpha \nabla_\theta J(\theta) = \alpha \mathbf{E}_\pi \left[(v_\pi(s) - V_\theta(s)) \nabla_\theta V_\theta(s) \right]$$

where the **target** is $v_\pi(s)$.

- Target is return G_t in MC policy iteration, $R_{t+1} + \gamma V_\theta(s_{t+1})$ in TD(0).

Loss function is similar for action value functions. For example, for TD estimation,

$$\Delta\theta = -\alpha \nabla_\theta J(\theta) = \alpha (R_{t+1} + \gamma \max_a Q_\theta(s_{t+1}, a) - Q_\theta(s_t, a_t)) \nabla_\theta Q_\theta(s_t, a_t)$$

Deadly Triad: Naive Q-Learning using a parameterized Q function and a least-squares objective

$$\min_\theta \frac{1}{2} \left(Q_\theta(s, a) - (r(s, a) + \gamma \max_{a'} Q_\theta(s', a')) \right)^2$$

is off-policy, uses function approximation, and bootstrapping, which makes convergence and efficient learning extremely difficult.

Deep Q-Learning [2]: Introduces a **target network** Q_{θ^-} which is a copy of the **online network** Q_θ delayed by some amount which gives a stable estimation of the value function. Also uses an **experience replay buffer** that stores (s, a, r, s') tuples from many trajectories and allows for data decorrelation and improved sample efficiency. The TD target is given by

$$y^{\text{DQN}} = r + \gamma \max_{a'} Q_{\theta^-}(s', a').$$

- In practice, this is implemented as a neural network $Q_\theta : \mathcal{S} \to \mathbb{R}^{|\mathcal{A}|}$.

Double DQN [3]: Deep Q-Learning using the online network to select the next action but using the target network to evaluate it:

$$a^* = \underset{a}{\operatorname{argmax}} Q_\theta(s', a')$$
$$y^{\text{Double-DQN}} = R_{t+1} + \gamma Q_{\theta^-}(s', a^*)$$

Clipped Double DQN [4]: Use the minimum of target and online network to evaluate the Q function:

$$y^{\text{Clipped-Double-DQN}} = R_{t+1} + \min_{Q \in \{Q_\theta, Q_{\theta^-}\}} Q(s', a^*).$$

Prioritizing Replays [5]: Within the experience replay buffer, weight each experience by its "surprise" or DQN error:

$$p_i = r_i + \gamma \max_a Q_{\theta^-}(s_i', a) - Q_\theta(s_i', a)$$
$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}.$$

- α parameterizes how much the priority is used, with $\alpha = 0$ being the uniform case.

Multistep Q-Learning:

$$I = (R_t^{(n)} + \gamma_t^{(n)} \max_{a'} Q_{\theta^-}(S_{t+n}, a') - Q_\theta(s, a))^2.$$

3 Evolutionary Methods

Policy optimization: policy optimization methods search over policy parameters without computing state or action values:

$$\max_\theta U(\theta) = \mathbf{E}_{\tau \sim \pi_\theta}\left[R(\tau) \mid \pi_\theta, \mu_0(s_0)\right].$$

Univariate Gaussian Distribution:

$$\mathcal{N}(y \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\frac{-(y-\mu)^2}{2\sigma^2}\right).$$

Multivariate Gaussian Distribution:

$$\mathcal{N}(y \mid \mu, \Sigma) = \frac{1}{(2\pi)^{d/2}\,|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(y-\mu)^\top \Sigma^{-1}(y-\mu)\right).$$

Cross-entropy Method: Initialize $\mu_0 \leftarrow 0$, $\Sigma_0 \leftarrow I$. For each step, sample $\theta_i \in \mathcal{N}(\mu, \sigma^2 I)$ for $i = 1, \ldots, N$. For each, evaluate over L trials and take top $\lceil \rho N \rceil$ θ_i by mean overall reward. Then, update via

$$\mu_t \leftarrow \frac{1}{K}\sum_{i \in \varepsilon} \theta_i$$
$$\Sigma_t \leftarrow \frac{1}{K}\sum_{i \in \varepsilon} (\theta_i - \mu_{t+1})(\theta_i - \mu_{t+1})^\top.$$

- Optionally, add smoothing $\mu_t \leftarrow (1-\alpha)\mu_t + \alpha\mu_{t-1}$, $\Sigma_t \leftarrow (1-\alpha)\Sigma_t + \alpha\Sigma_{t-1}$.

Natural Evolutionary Strategy: Maximize

$$\max_{\mu} \mathbf{E}_{\theta \sim P_\mu(\theta)} [F(\theta)]$$

where $P_\mu(\theta) = \mathcal{N}(\mu, \sigma^2 I)$ (σ^2 is a constant hyperparameter) and $F(\theta) = \mathbf{E}_{\tau \sim \pi_0, s_0 \sim \mu_0(s)} [R(\tau)]$. The corresponding gradient update is

$$\begin{aligned}
\nabla_\mu \mathbf{E}_{\theta \sim P_\mu(\theta)} [F(\theta)] &= \nabla_\mu \int P_\mu(\theta) F(\theta) \, \mathrm{d}\theta \\
&= \int P_\mu(\theta) \cdot \frac{\nabla_\mu P_\mu(\theta)}{P_\mu(\theta)} F(\theta) \, \mathrm{d}\theta \\
&= \int P_\mu(\theta) \cdot \nabla_\mu \log P_\mu(\theta) F(\theta) \, \mathrm{d}\theta \\
&= \mathbf{E}_{\theta \sim P_\mu(\theta)} [\nabla_\mu \log P_\mu(\theta) F(\theta)] \\
&\approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\mu \log P_\mu(\theta)|_{\theta=\theta_i} F(\theta_i)
\end{aligned}$$

where $\theta_i \sim P_\mu(\theta)$. We also have that

$$\nabla_\mu \log P_\mu(\theta) = \nabla_\mu - \frac{\|\theta - \mu\|^2}{2\sigma^2} = \frac{\theta - \mu}{\sigma^2}.$$

We can express $\theta = \mu + \sigma\epsilon$ for $\epsilon \sim \mathcal{N}(0, 1)$, so the gradient becomes

$$\nabla_\mu \mathbf{E}_{\theta \sim P_\mu(\theta)} [F(\theta)] \approx \frac{1}{N\sigma} \sum_{i=1}^{N} \epsilon_i F(\mu + \sigma\epsilon_i).$$

Distributed Evolution: Server broadcasts seeds $s_1, \ldots, s_k$; each core $i \in [k]$ computes ϵ_j from seed s_i and broadcasts aggregate result back to the server.

4 Policy-Gradient Methods

Policy Optimization: For some objective function $U(\theta)$, sample trajectories $\tau_i = \{s_t^i, a_t^i\}_{t=0}^T$ by deploying π_θ and then update $\theta \leftarrow \theta + \alpha \nabla_\theta U(\theta)$.

Policy Objectives: The parameterization of the network depends on the type of task:

- Deterministic discrete/continuous: $a = \pi_\theta(s)$
- Stochastic continuous: $a \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta^2(s))$
- Stochastic discrete: $a \sim \mathrm{softmax}(\cdot)$.

In general, a reasonable policy objective is

$$\max_\theta U(\theta) = \mathbf{E}_{\tau \sim P(\tau;\theta)} [R(\tau)] = \sum_\tau P(\tau;\theta) R(\tau)$$

$$P(\tau;\theta) = \rho_0(s_0) \prod_{t=0}^{H} \underbrace{P(s_{t+1} \mid a_t, s_t)}_{\text{dynamics}} \underbrace{\pi_\theta(a_t \mid s_t)}_{\text{policy}}$$

In this form, backpropogation is not directly applicable because to get $\nabla_\theta P(\tau;\theta)$ requires knowing the dynamics of the environment.

- The gradiant can be numerically approximated, but this is not feasible for high-dimensional parameter spaces:
$$\frac{\partial U(\theta)}{\partial \theta_k} \approx \frac{U(\theta + \epsilon u_k) - U(\theta - \epsilon u_k)}{2\epsilon}.$$

4.1 REINFORCE and Actor-Critic Algorithms

General Policy Score: Consider the following expression for $P(\cdot)$ continuous, differentiable, and easy to sample:

$$\begin{aligned}
\nabla_\theta \mathbf{E}_{x\sim P_\theta(x)}\left[f(x)\right] &= \nabla_\theta \int P_\theta(x) f(x)\ \mathrm{d}x \\
&= \int \nabla_\theta P_\theta(x) f(x)\ \mathrm{d}x \\
&= \int \frac{\nabla_\theta P_\theta(x)}{P_\theta(x)} P_\theta(x) f(x)\ \mathrm{d}x \\
&= \int P_\theta(x) \cdot \nabla_\theta \log P_\theta(x) f(x)\ \mathrm{d}x \\
&= \mathbf{E}_{x\sim P_\theta(x)}\left[\nabla_\theta \log P_\theta(x) f(x)\right] \\
&\approx \frac{1}{N}\sum_{i=1}^{N} \nabla_\theta \log P_\theta(x^{(i)}) f(x^{(i)}).
\end{aligned}$$

This form is easier to work with because one only needs to evaluate $\nabla_\theta \log P_\theta(x)$ for x sampled from $P_\theta(x)$.

- Note that we still need trajectories $\tau^{(i)} \sim P_\theta$ to remain on-policy.
- This expression can be interpreted as a weighted sum pulling the gradient towards directions with higher rewards.

Policy Scores for Gaussian and Softmax Policies: We have that

$$\begin{aligned}
\nabla_\theta \log P_\theta(\tau) &= \nabla_\theta \log\left(\prod_{t=0}^{T} P(s_{t+1}^{(i)} \mid s_t^{(i)}, a_t^{(i)}) \pi_\theta(a_t^{(i)} \mid s_t^{(i)})\right) \\
&= \nabla_\theta \sum_{t=0}^{T} \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \\
&= \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)})
\end{aligned}$$

- For Gaussian policies,
$$\nabla_\theta \log \pi_\theta(a \mid s) \propto (\Sigma^{-1}(a - \mu_\theta(s)))^\top \frac{\partial(\mu_1, \ldots, \mu_m)}{\partial(\theta_1, \ldots, \theta_n)}$$
- For softmax policies,
$$\nabla_\theta \log \pi_\theta(a \mid s) = \nabla_\theta h_\theta(s, a) - \sum_{a'} \pi_\theta(a' \mid s) \nabla_\theta h_\theta(s, a'),$$

where $\pi_\theta(a \mid s) = \exp(h_\theta(s,a))/\sum_{a'}(\cdot)$.

Cold-Start Problem: For trajectories with 0 reward, policy just randomly explores.

REINFORCE Algorithm [6]: Combining the two previous derivations with reward function $f(x) = G_t$, we get

$$\hat{g} = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)})G_t^{(i)}$$

$$\theta_{t+1} \leftarrow \theta_t + \alpha\gamma^t G_t \frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}$$

Compared with the MC policy iteration method, REINFORCE samples over parameter space instead of action space.

State-Dependent Baseline: A state-based baseline may be subtracted from the REINFORCE gradient without changing the gradient:

$$\sum_{t=0}^{H}(G_t - b(s_t))\nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

since $\mathbf{E}_{a\sim\pi_\theta(\cdot|s)}\left[\nabla_\theta \log \pi_\theta(a \mid s)\right] = 0$. Setting $b(s_t) = v_\pi(s)$ reduces variance:

$$\mathrm{Va}(\hat{g}) = \operatorname{tr}\mathbf{E}\left[(\hat{g} - \mathbf{E}\left[\hat{g}\right])(\hat{g} - \mathbf{E}\left[\hat{g}\right])^\top\right] = \sum_{i=1}^{n}\mathbf{E}\left[(\hat{g}_k - \mathbf{E}\left[\hat{g}_k\right])^2\right].$$

- Motivation: We care about the advantage of an action conditioned on a state, not only when it has high returns:
$$G_t - v_\pi(s_t) \approx q_\pi(s_t, a_t) - v_\pi(s_t) = A_\pi(s_t, a_t).$$

Actor-Critic Algorithm [7]: Let $V_\phi^\pi(s_t)$ parameterized by ϕ approximate $v_\pi(s_t)$ from trajectories derived from π. The approximate value function can be computed at each step using standard value-based approaches:

- Monte-Carlo:
$$\phi \leftarrow \operatorname*{argmin}_\phi \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\left(V_\phi^\pi(s_t^{(i)} - G_t^{(i)})\right)^2.$$
- TD(0):
$$\phi \leftarrow \operatorname*{argmin}_\phi \sum_{(s,a,r,s')}\|r + \gamma V_\phi^\pi(s') - V_\phi^\pi(s)\|_2^2$$

The pseudocode for actor-critic:

(1) Sample $\tau_i = \left\{s_t^{(i)}, a_t^{(i)}\right\}_{t=1}^{T}$ by deploying policy π_θ.

(2) Update ϕ by fitting V_ϕ^π with MC/TD(0) methods.

(3) Compute advantages $A^\pi(s_t^{(i)}, a_t^{(i)}) = G_t^{(i)} - V_\phi^\pi(s_t^{(i)})$.

(4) Update θ via

$$\nabla_\theta U(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) A_\pi(s_t^{(i)}, a_t^{(i)})$$
$$\theta \leftarrow \theta + \alpha \nabla_\theta U(\theta).$$

Advantage Actor-Critic (A2C) [8]: We can express $A_\pi(s_t^{(i)}, a_t^{(i)}) = Q_\phi^\pi(s_t^{(i)}, a_t^{(i)}) - V_\phi^\pi(s_t^{(i)})$ where $Q_\phi^\pi(s_t^{(i)}, a_t^{(i)}) = R(s_t^{(i)}, a_t^{(i)}) + \gamma V_\phi^\pi(s_{t+1}^{(i)})$.

High Data Correlation: Trajectory samples taken in actor-critic algorithms are high correlated, but stochastic gradient requires uncorrelated, uniformly sampled data. Furthermore, actor-critic algorithms are required to be on-policy (so experience buffer does not work). This can be solved in practice by running many RL environments in parallel.

Entropy Regularization: The **Shannon entropy** of a state-conditioned policy is given by

$$\mathcal{H}(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s)\log \pi(a \mid s) = -\mathbf{E}_{a\sim\pi(\cdot|s)}\left[\log \pi(a \mid s)\right].$$

We add this term to the actor loss:

$$U(\theta) = \mathbf{E}_{\tau\sim P(\tau;\theta)}\left[R(\tau) - \beta\mathcal{H}(\pi_\theta(\cdot \mid s_t))\right].$$

This can be differentiated via

$$\nabla_\theta \mathcal{H}(\pi_\theta(a \mid s)) = -\mathbf{E}_{a\sim\pi_\theta(\cdot|s)}\left[\nabla_\theta \log \pi_\theta(a \mid s)(\log \pi_\theta(a \mid s) + 1)\right].$$

Update-to-Data (UTD) Ratio: The update-to-data ratio is defined as the number of gradient update divided by the number of new environment transitions. A high UTD leads to being too off-policy and makes it unable to escape local minimums, whereas a low UTD is data inefficient.

4.2 PPO

Performance Difference Lemma:

$$J(\pi') - J(\pi) = \mathbf{E}_{s\sim d^{\pi'}, a\sim\pi'(\cdot|s)}\left[A_\pi(s,a)\right].$$

Proof: We have that

$$\begin{aligned}\mathbf{E}_{a\sim\pi'}\left[A_\pi(s,a)\right] &= \mathbf{E}_{a\sim\pi'}\left[q_\pi(s,a) - v_\pi(s)\right]\\ &= \mathbf{E}_{a\sim\pi'}\left[q_\pi(s,a)\right] - v_\pi(s)\\ &= \mathbf{E}_{a\sim\pi'}\left[r(s,a)\right] + \gamma\mathbf{E}_{a\sim\pi', s'\sim P(\cdot|s,a)}\left[v_\pi(s',a)\right] - v_\pi(s).\end{aligned}$$

For fixed (s_t, a_t) from π' with $s_0 \sim \rho_0$, we have

$$\begin{aligned}
\mathbf{E}_{\pi'}\left[A_\pi(s_t, a_t)\right] &= \mathbf{E}_{\pi'}\left[r_t\right] + \gamma \mathbf{E}_{\pi'}\left[v_\pi(s_{t+1})\right] - \mathbf{E}_{\pi'}\left[v_\pi(s_t)\right] \\
\sum_{t=0}^{\infty} \gamma^t \mathbf{E}_{\pi'}\left[A_\pi(s_t, a_t)\right] &= \sum_{t=0}^{\infty} \gamma^t \left(\mathbf{E}_{\pi'}\left[r_t\right] + \gamma \mathbf{E}_{\pi'}\left[v_\pi(s_{t+1})\right] - \mathbf{E}_{\pi'}\left[v_\pi(s_t)\right]\right) \\
&= \left(\sum_{t=0}^{\infty} \gamma^t \mathbf{E}_{\pi'}\left[r^t\right]\right) - \mathbf{E}_{s_0 \sim \rho_0}\left[v_\pi(s_0)\right] \\
&= J(\pi') - J(\pi).
\end{aligned}$$

By definition, we have

$$\mathbf{E}_{s \sim d^{\pi'}, a \sim \pi'(\cdot|s)}\left[A_\pi(s, a)\right] = \sum_{t=0}^{\infty} \gamma^t \mathbf{E}_{\pi'}\left[A_\pi(s_t, a_t)\right],$$

concluding the proof.

Importance Sampling: Consider the problem of computing $\mathbf{E}_{z \sim p}\left[f(z)\right] = \int f(z) p(z) \, \mathrm{d}z$ but $p(z)$ cannot be sampled directly. Instead, a **proposal distribution** $q(z)$ can be sampled from. We have that

$$\mathbf{E}_{z \sim p}\left[f(z)\right] = \mathbf{E}_{z \sim q}\left[\frac{p(z)}{q(z)} f(z)\right] \approx \frac{1}{N} \sum_{i=1}^{N} \underbrace{\frac{p(z^{(i)})}{q(z^{(i)})}}_{\text{importance weight}} f(z^{(i)})$$

Importance-Weighted Policy Gradients: Policy optimization reduces to

$$\max_{\pi'} J(\pi') = \max_{\pi'} J(\pi') - J(\pi) = \max_{\pi'} \mathbf{E}_{s \sim d^{\pi'}} \mathbf{E}_{a \sim \pi'(\cdot|s)}\left[A_\pi(s, a)\right]$$

We approximate $d^\pi(s) \approx d^{\pi'}(s)$ so that we can sample states from the existing policy π and apply the distribution shift correction from $\pi'(\cdot \mid s)$ to $\pi(\cdot \mid s)$:

$$\mathbf{E}_{s \sim d^{\pi'}} \mathbf{E}_{a \sim \pi'(\cdot|s)}\left[A_\pi(s, a)\right] = \mathbf{E}_{s \sim d^{\pi'}} \mathbf{E}_{a \sim \pi(\cdot|s)}\left[\frac{\pi'(a \mid s)}{\pi(a \mid s)} A_\pi(s, a)\right] \approx \mathbf{E}_{s \sim d^{\pi}} \mathbf{E}_{a \sim \pi(\cdot|s)}\left[\frac{\pi'(a \mid s)}{\pi(a \mid s)} A_\pi(s, a)\right]$$

We have that

$$\nabla_{\theta'} \mathbf{E}_{s \sim d^{\pi}} \mathbf{E}_{a \sim \pi(\cdot|s)}\left[\frac{\pi'(a \mid s)}{\pi(a \mid s)} A_\pi(s, a)\right] = \mathbf{E}_{s \sim d^{\pi}} \mathbf{E}_{a \sim \pi(\cdot|s)}\left[\nabla_{\theta'} \log \pi'(a \mid s) \frac{\pi'(a \mid s)}{\pi(a \mid s)} A_\pi(s, a)\right]$$

(recall that $\nabla_\theta \log \pi(a \mid s)$ has a closed-form solution for Gaussian and softmax policies). To account for the approximation, we add a penalty term to the loss to make it a **constrained maximization** problem:

$$\max_{\pi'}(\cdot) - \alpha \mathbf{E}_s\left[\mathcal{D}_{\mathsf{KL}}(\pi'(\cdot \mid s) \| \pi(\cdot \mid s))\right].$$

Proximity Policy Optimization (PPO) [9]: Gradient is "clipped" (reduced to 0) if $\pi'(a \mid s)$ diverges from $\pi(a \mid s)$.

$$\max_{\pi'} \mathbf{E}_{s \sim d^{\pi}(\cdot)} \mathbf{E}_{a \sim \pi(\cdot|s)} \mathrm{clip}\left(\frac{\pi'(a \mid s)}{\pi(a \mid s)}, 1 - \epsilon_-, 1 + \epsilon_+\right) A_\pi(s, a)$$

For example, if $A_\pi > 0$, then $\pi'(a \mid s) > \pi(a \mid s)$. Applying this update iteratively will clip it to $1 + \epsilon$. Alternatively,

$$\max_{\pi'} \mathbf{E}_{s\sim d^\pi(\cdot)} \mathbf{E}_{a\sim\pi(\cdot|s)} \min\left\{ \frac{\pi'(a \mid s)}{\pi(a \mid s)} A_\pi(s,a), \text{clip}\left(\frac{\pi'(a \mid s)}{\pi(a \mid s)}, 1-\epsilon_-, 1+\epsilon_+ \right) A_\pi(s,a) \right\}$$

addresses the problem of if $A_\pi > 0$ but $\frac{\pi'(a|s)}{\pi(a|s)} < 1 - \varepsilon_-$.

- Typically, $\varepsilon_+ > \varepsilon_-$ to encourage exploration by emphasizing good actions.

4.3 TRPO

Natural Gradient Decent: A form of gradient decent that considers divergence in distribution space, instead of Euclidean distance in parametr space:

$$\begin{aligned} \text{gradient decent:} d^* &= \underset{\|d\|_2<\epsilon}{\text{argmax}}\, U(\theta + d) \\ \text{natural gradient decent:} d^* &= \underset{\mathcal{D}_{\text{KL}}(\pi_\theta \| \pi_{\theta+d})<\epsilon}{\text{argmax}}\, U(\theta + d) \end{aligned}$$

where $\mathcal{D}_{\text{KL}}$ is the Knullback-Leibler divergence defined by

$$\mathcal{D}_{\text{KL}}(P\|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}.$$

KL-Constrained Objective / Derivation of TRPO:

$$\begin{aligned} &\max_\theta \overline{\mathbb{A}}_{\pi_{\text{old}}}(\pi) = \sum_{t=1}^T \mathbf{E}_{s_t\sim P_{\theta_{\text{old}}}(\cdot)} \mathbf{E}_{a_t\sim\pi_{\theta_{\text{old}}}(\cdot|s_t)} \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} A_{\pi_{\theta_{\text{old}}}}(s_t, a_t) \quad \text{such that} \\ &\mathbf{E}_t\left[\mathcal{D}_{\text{KL}}\left(\pi_{\theta_{\text{old}}}(\cdot \mid s_t) \| \pi_\theta(\cdot \| s_t) \right) \right] \le \delta \end{aligned}$$

For the unconstrained objective:

$$\begin{aligned} \nabla_\theta \bar{\mathbb{A}}_{\pi_{\text{old}}}(\pi)|_{\theta=\theta_{\text{old}}} &= \sum_{t=1}^T \mathbf{E}_{s_t\sim p_{\theta_{\text{old}}}(s_t)} \mathbf{E}_{a_t\sim\pi_{\theta_{\text{old}}}(a_t|s_t)} \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)|_{\theta=\theta_{\text{old}}} A_{\pi_{\theta_{\text{old}}}}(s_t, a_t) \right] \\ \nabla_\theta \bar{\mathbb{A}}_{\pi_{\text{old}}}(\pi)|_{\theta=\theta'} &= \sum_{t=1}^T \mathbf{E}_{s_t\sim p_{\theta_{\text{old}}}(s_t)} \mathbf{E}_{a_t\sim\pi_{\theta_{\text{old}}}(a_t|s_t)} \left[\frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \nabla_\theta \log \pi_\theta(a_t \mid s_t)|_{\theta=\theta'} A_{\pi_{\theta_{\text{old}}}}(s_t, a_t) \right] \end{aligned}$$

Consider the unconstrained penalized objective:

$$\begin{aligned} d^* &= \underset{d}{\text{argmax}}\, U(\theta + d) - \lambda \mathcal{D}_{\text{KL}}(\pi_\theta \| \pi_{\theta+d} - \delta) \\ &\approx \underset{d}{\text{argmax}}\, U(\theta_{\text{old}}) + \nabla_\theta U(\theta)|_{\theta=\theta_{\text{old}}} \\ &\quad - \lambda \left[\mathcal{D}_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_{\theta_{\text{old}}}) + d^\top \nabla_\theta \mathcal{D}_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta)|_{\theta=\theta_{\text{old}}} + \frac{1}{2}\lambda \left(d^\top \nabla_\theta^2 \mathcal{D}_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta)|_{\theta=\theta_{\text{old}}} d \right) \right] + \lambda\delta \end{aligned}$$

using the first-order Taylor expansion for the objective and second-order Tayler expansion for the KL. Clearly, $\mathcal{D}_{\mathsf{KL}}(\pi_{\theta_{\mathsf{old}}} \| \pi_{\theta_{\mathsf{old}}}) = 0$ and $d^\top \nabla_\theta \mathcal{D}_{\mathsf{KL}}(\pi_{\theta_{\mathsf{old}}} \| \pi_\theta)|_{\theta=\theta_{\mathsf{old}}} = 0$ because the KL-divergence achieves a global minimum at $\pi_\theta = \pi_{\theta_{\mathsf{old}}}$. We have that

$$\begin{aligned}
\nabla_\theta^2 \mathcal{D}_{\mathsf{KL}}(\pi_{\theta_{\mathsf{old}}} \| \pi_\theta)|_{\theta=\theta_{\mathsf{old}}} &= -\mathbf{E}_{x\sim P_{\theta_{\mathsf{old}}}} \nabla_\theta^2 \log P_\theta(x)|_{\theta=\theta_{\mathsf{old}}} \\
&= -\mathbf{E}_{x\sim P_{\theta_{\mathsf{old}}}} \nabla_\theta \left(\frac{\nabla_\theta P_\theta(x)}{P_\theta(x)} \right) |_{\theta=\theta_{\mathsf{old}}} \\
&= -\mathbf{E}_{x\sim P_{\theta_{\mathsf{old}}}} \left(\frac{\nabla_\theta^2 P_\theta(x) P_\theta(x) - \nabla_\theta P_\theta(x) \nabla_\theta P_\theta(x)^\top}{P_\theta(x)^2} \right) |_{\theta=\theta_{\mathsf{old}}} \\
&= -\mathbf{E}_{x\sim P_{\theta_{\mathsf{old}}}} \frac{\nabla_\theta^2 P_\theta(x)|_{\theta=\theta_{\mathsf{old}}}}{P_{\theta_{\mathsf{old}}}(x)} + \mathbf{E}_{x\sim P_{\theta_{\mathsf{old}}}} \nabla_\theta \log P_\theta(x) \nabla_\theta \log P_\theta(x)^\top |_{\theta=\theta_{\mathsf{old}}} \\
&= \underbrace{\mathbf{E}_{x\sim P_{\theta_{\mathsf{old}}}} \nabla_\theta \log P_\theta(x)|_{\theta=\theta_{\mathsf{old}}} \nabla_\theta \log P_\theta(x)|_{\theta=\theta_{\mathsf{old}}}^\top}_{\mathbf{F}(\theta)\text{:Fisher information matrix}} \\
&\approx \frac{1}{N} \sum_{i=1, x^{(i)}\sim p_{\theta_{\mathsf{old}}}}^{N} [\nabla_\theta \log P_\theta(x)|_{\theta=\theta_{\mathsf{old}}} \nabla_\theta \log P_\theta(x)|_{\theta=\theta_{\mathsf{old}}}]
\end{aligned}$$

Thus the gradient approximation reduces to

$$\begin{aligned}
d^* &\approx \operatorname*{argmax}_d \nabla_\theta U(\theta)|_{\theta=\theta_{\mathsf{old}}} \cdot d - \frac{1}{2} \lambda d^\top \mathbf{F}(\theta) d \\
&= \operatorname*{argmin}_d -\nabla_\theta U(\theta)|_{\theta=\theta_{\mathsf{old}}} \cdot d + \frac{1}{2} \lambda d^\top \mathbf{F}(\theta) d.
\end{aligned}$$

Setting the gradient to 0 gives

$$\begin{aligned}
0 &= \frac{\partial}{\partial x} \left(-\nabla_\theta U(\theta)|_{\theta=\theta_{\mathsf{old}}} \cdot d + \frac{1}{2} \lambda d^\top \mathbf{F}(\theta) d \right) \\
&= -\nabla_\theta U(\theta)|_{\theta=\theta_{\mathsf{old}}} + \frac{1}{2} \lambda d^\top \mathbf{F}(\theta) \\
d &= \frac{2}{\lambda} \mathbf{F}^{-1}(\theta_{\mathsf{old}}) \nabla_\theta U(\theta)|_{\theta=\theta_{\mathsf{old}}}
\end{aligned}$$

Because $2/\lambda$ is a constant, the gradient update is simply $g_N = \mathbf{F}^{-1}(\theta_{\mathsf{old}}) \nabla_\theta U(\theta)|_{\theta=\theta_{\mathsf{old}}}$ where the second term is the policy gradient $\nabla_\theta \log \pi_\theta(a \mid s) A_\pi(s, a)$:

$$\theta_{\mathsf{new}} \leftarrow \theta_{\mathsf{old}} + \alpha \cdot g_N$$

To constrain the step size α, we approximate

$$\mathcal{D}_{\mathsf{KL}}(\pi_{\theta_{\mathsf{old}}} \| \pi_\theta) \approx \frac{1}{2} (\theta - \theta_{\mathsf{old}})^\top \mathbf{F}(\theta_{\mathsf{old}}) (\theta - \theta_{\mathsf{old}}) = \frac{1}{2} (\alpha g_N)^\top \mathbf{F} (\alpha g_N) = \delta.$$

This gives

$$\alpha = \sqrt{\frac{2\delta}{g_N^\top \mathbf{F}^{-1} g_N}}.$$

Trust Region Policy Optimization (TRPO) [10]: Because the KL divergence bound was derived from the second order Tayler approximation, it may be violated in practice. We can line search for a step size in $\{\alpha^0, \ldots, \alpha^L\}$ for the first index s.t. $\overline{\mathbb{A}}_{\pi_{\mathsf{old}}}(\pi) \geq 0$ and $\mathcal{D}_{\mathsf{KL}}(\theta_{\mathsf{new}} \| \theta) \leq \delta$.

4.4 Path-Wise Policy Gradients

Estimation of Q-functions Off-Policy: The action-value function can be estimated from any policy:

$$q_\pi(s,a) = r(s,a) + \gamma \mathbf{E}_{s'\sim P(\cdot|s,a)} \mathbf{E}_{a'\sim\pi(\cdot|s')} q_\pi(s',a')$$

where the TD estimation loss of q_π is given by

$$\mathcal{L}(\phi) = \mathbf{E}_{(s,a,r,s')\sim\text{Replay-Buffer}} \left[\left(Q_\phi(s,a) - y(s,a) \right)^2 \right]$$

with target

$$y(s,a) = r(s,a) + \gamma \mathbf{E}_{s'\sim P(\cdot|s,a)} \mathbf{E}_{a'\sim\pi(\cdot|s')} Q_{\phi,\text{tgt}}(s',a').$$

The target network can be updated with

- **hard updates**: $\theta^- \leftarrow \theta$ every C steps
- **soft updates**: $\phi^{\text{tgt}} \leftarrow \tau\phi + (1-\tau)\phi^{\text{tgt}}$ for $\tau \in [0,1]$.

Policy Objectives from Q-functions: We can reframe the policy objective

$$\max_\theta J(\theta) = \max_\theta \mathbf{E}_{\tau\sim P_\theta(\cdot)} \left[\sum_{t=1}^{T} Q_\phi(s_t,a_t) \right]$$

where θ paramerizes the policy π_θ and Q_ϕ is the estimated action-value function. If the actor is deterministic (action $a = \mu_\theta(s)$), then

$$\mathbf{E} \sum_{t=0}^{T} \frac{\mathrm{d}}{\mathrm{d}\theta} Q_\phi(s_t,a_t) = \mathbf{E} \sum_{t=0}^{T} \frac{\partial Q_\phi(s_t,a)}{\partial a}|_{a=a_t} \frac{\partial a_t}{\partial \theta}$$

Computing Gradients of Expectations: If the differentiated variable appears in the expectation, then

$$\partial_\theta \mathbf{E}_{x\sim P(x)} \left[f(x(\theta)) \right] = \mathbf{E}_{x\sim P(x)} \left[\partial\theta f(x(\theta)) \right] = \mathbf{E}_{x\sim P(x)} \left[\frac{\mathrm{d} f(x(\theta))}{\mathrm{d}x} \frac{\mathrm{d}x}{\mathrm{d}\theta} \right].$$

If the differentiated variable appears in the distribution, then we can either use the **likelihood ratio gradient estimator**

$$\partial_\theta \mathbf{E}_{x\sim P_\theta(\cdot)} \left[f(x) \right] = \mathbf{E}_{x\sim P_\theta(\cdot)} \left[\partial_\theta \log P_\theta(x) f(x) \right]$$

or **Reparameterize for Gaussian distributions**:

$$\partial_\theta \mathbf{E}_{z\sim\mathcal{N}(0,1)} \left[f(x(z,\theta)) \right] = \mathbf{E}_{z\sim\mathcal{N}(0,1)} \left[\frac{\mathrm{d}f}{\mathrm{d}x} \left(\frac{\mathrm{d}\mu(\theta)}{\mathrm{d}\theta} + z \frac{\mathrm{d}\sigma(\theta)}{\mathrm{d}\theta} \right) \right].$$

Deep Deterministic Policy Gradients (DDPG) Actor-Critic [11]: DDPG target:

$$a'_{t+1} \leftarrow \pi_\theta(s_{t+1})$$
$$y_t^{\text{DDPG}} \leftarrow r + \gamma Q_{\bar{\phi}}(s_{t+1}, a'_{t+1})$$

(1) Sample $\tau_i = \left\{ s_t^{(i)}, a_t^{(i)} \right\}_{t=1}^{T}$ by deploying policy π_θ with some exploration noise. Add these to a replay buffer.

(2) Update value function by regressing to targets y_t^{DDPG} (similar to TD(0))

(3) Update policy $\theta \leftarrow \theta + \alpha \nabla_a Q_\phi(s,a)|_{a=\pi_\theta(s)} \nabla_\theta \pi_\theta(s)$

(4) Update the parameters $\overline{\theta} \leftarrow \tau\theta + (1-\tau)\overline{\theta}$ and $\overline{\phi} \leftarrow \tau\phi + (1-\tau)\overline{\phi}$

- Issues with DDPG: overestimation of Q-values, high variance in target estimates, sensitivity to hyperparameters, unstable policy updates.

Reparameterized Policy Gradients: For Gaussian policies where θ parameterizes $a \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta^2(s))$ we can reparameterize them as $a = \mu_\theta(s) + \sigma_\theta(s) \odot z$ for $z \sim \mathcal{N}(0,1)$:

$$\mathbf{E}\sum_{t=0}^{T} \frac{\mathrm{d}}{\mathrm{d}\theta} Q_\phi(s_t, a_t) = \mathbf{E}\sum_{t=0}^{T} \frac{\partial Q_\phi(s_t,a)}{\partial a}|_{a=a_t} \frac{\partial a_t}{\partial \theta} = \mathbf{E}\sum_{t=0}^{T} \frac{\partial Q_\phi(s_t,a)}{\partial a}|_{a=a_t} \left(\frac{\mathrm{d}\mu_\theta(s_t)}{\mathrm{d}\theta} + z_t \frac{\mathrm{d}\sigma_\theta(s_t)}{\mathrm{d}\theta} \right)$$

Twin Delayed DDPG (TD3) [4]: Uses clipped double Q-learning with target policy smoothing and delayed policy updates. The TD3 critic target is

$$\begin{aligned} a'_{t+1} &\leftarrow \pi_{\overline{\theta}}(s_{t+1}) + \epsilon \\ y_t^{\text{TD3}} &\leftarrow r_t + \gamma \min_{i \in \{1,2\}} Q_{\overline{\phi}_i}(s_{t+1}, a'_{t+1}) \\ \phi_i &\leftarrow \phi_i - \alpha \nabla_{\phi_i} \mathbf{E}_{\mathcal{D}} \left[\left(Q_{\phi_i}(s_t,a_t) - y_t^{\text{TD3}} \right)^2 \right] \end{aligned}$$

where $\epsilon \sim$ clipped $\mathcal{N}(0, \sigma^2)$ is a noise vector. Delayed policy updates means that the actor network is updated less frequently than the critic network (1:2 ratio). The Q-functions are also allowed to converge for a fixed policy to allow for more stable targets.

4.5 Soft Policy Gradient Methods

Principle of Maximum Entropy: Policies that generate equal rewards should be equally probable.

Maximum Entropy RL Objective: To promote stochastic policies, we can add an entropy term:

$$\begin{aligned} \pi^* &= \underset{\pi}{\operatorname{argmax}}\, \mathbf{E}_{\tau\sim\pi} \left[\sum_{t=1}^{T} \gamma^t r(s_t,a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t)) \right] \\ &= \underset{\pi}{\operatorname{argmax}}\, \mathbf{E}_{\tau\sim\pi} \left[\sum_{t=1}^{T} \gamma^t r(s_t,a_t) + \alpha \mathbf{E}_{a\sim\pi(\cdot|s)} \left[-\log \pi(a \mid s_t) \right) \right] \\ &= \underset{\pi}{\operatorname{argmax}}\, \mathbf{E}_{\tau\sim\pi} \left[\sum_{t=1}^{T} \gamma^t r(s_t,a_t) - \alpha \log \pi(a_t \mid s_t) \right] \end{aligned}$$

- The maximum entropy policy is stochastic, whereas the optimal policy in an MDP is deterministic. This encourages exploration in training.
- This differs from **entropy-regularized policy gradient** because adding an entropy term in the gradient only maximizes the entropy of the policy in the current timestep, whereas adding it to the RL objective makes the agent seek future states with high entropy.
  - **Backing up** entropy allows agent to succeed even when the action-space is noisy.

Soft Bellman Equations: Total reward obtained by following action a in state s and backing up the entropy after:

$$q_\pi^{\text{soft}}(s,a) = r(s,a) + \gamma \mathbf{E}_{s',a' \sim \pi(\cdot|s')} \left[q_\pi^(s',a') - \alpha \log \pi(a' \mid s') \right].$$

This gives

$$q_\pi^{\text{soft}}(s,a) = r(s,a) + \sum_{s'} P(s' \mid s,a) \sum_{a'} \left(q_\pi^{\text{soft}}(s',a') - \alpha \log \pi(a' \mid s') \right).$$

Soft Policy Iteration [12]: An algorithm similar to policy-gradient iteration for soft policies. The **soft policy evaluation step** updates

$$q_\pi^{\text{soft}}(s,a) = r(s,a) + \gamma \mathbf{E}_{s',a' \sim \pi(\cdot|s')} \left[q_\pi^(s',a') - \alpha \log \pi(a' \mid s') \right].$$

and the **soft policy improvement step** updates

$$\pi(\cdot \mid s) = \operatorname*{argmax}_\pi \mathbf{E}_{a|\pi(\cdot|s)} \left[q_\pi^{\text{soft}}(s,a) - \alpha \log \pi(a \mid s) \right].$$

or equivalently,

$$\phi^* = \operatorname*{argmax}_\phi \mathbf{E}_{s,a \sim \pi_\phi(\cdot|s)} \left[Q_\theta(s,a) - \alpha \log \pi_\phi(a \mid s) \right]$$

Soft Actor Critic (SAC) [12]: Combines soft policy-improvement with ideas from DDPG and TD3. Uses double Q-learning and a replay buffer, with entropy regularization in both the actor and critic losses.

5 Imitation Learning

Behavior Cloning (BC) [13]: Given an expert trajectory $\tau = (s_0, a_0, r_1, s_1, \dots)$ from an expert π^*, train on supervised objective to match expert:

$$\theta^* = \operatorname*{argmin}_\theta \mathbf{E}_{(s_t,a_t) \sim \pi^*} \left[-\log \pi_\theta(a_t \mid s_t) \right].$$

If π_θ parameterizes a Gaussian $\mathcal{N}(\mu_\theta(s_t), \Sigma)$, then this reduces to the least squares objective

$$\theta^* = \operatorname*{argmin}_\theta \mathbf{E}_{(s_t,a_t) \sim \pi^*} \left[\|a_t - \mu_\theta(s_t)\|^2 \right].$$

Distribution Mismatch: Distributions in training/test sets are different ($p_{\pi^*}(s_t) \neq p_{\pi_\theta}(s_t)$). Solutions are to use data augmentation (create synthetic data in simulation), augment data collection, or query experts for additional data points.

- Experts can be RL policies (e.g. trained using policy gradient methods) with access to privileged information.

Data Multimodality: Expert state-conditioned action distributions often do not follow a Gaussian distribution (e.g. steering angle around an obstacle). Least-squares regression on neural networks fail to handle this case correctly.

5.1 Generative Adversarial Networks

Generative Adversarial Networks (GAN) [14]: A neural network comprised of a **generator** (sampler: noise vector $\sim \mathcal{N}(0,1) \to$ data)) and a **discriminator** (classifier: data $\to \{\text{clean}, \text{generated}\}$):

$$\min_G \max_D V(D,G) = \underbrace{\mathbf{E}_{x\sim P_{\text{data}}(x)}\left[\log D(x)\right]}_{\text{training set}} + \underbrace{\mathbf{E}_{z\sim P_z(x)}\left[\log(1-D(G(z)))\right]}_{\text{generated data}}$$

- Backpropagation for the generator involves computing gradients for discriminator (weights are frozen).
- If distribution of generated and train images, discriminator will achieve low loss.
- Discriminator is not trained to convergence to avoid overfitting.

Conditional GANs: Add input of condition (state in RL context) to both the generator and discriminator and find the policy π_θ that fools the discriminator from expert demonstration and trajectory.

- Using (s,a,r,s') tuples rather than trajectories makes discriminator less prone to overfitting.
- It is possible to train conditional GANs using only the state by training the discriminator on the state values or the transitions between adjacent states. This is useful when the action spaces of the expert and the agent are different.

Generative Adversarial Reinforcement Learning (GAIL) [15]: Using a conditional GAN to generate actions conditioned on states. This learning paradigm requires an environment.

$$\begin{aligned}\text{generator objective} &: \mathbf{E}_{(s,a)\sim\pi_\theta}\left[-\log D_\phi(s,a)\right] \\ \text{discriminator objective} &: \mathbf{E}_{(s,a)\sim\text{demo}}\left[\log(1-D_\phi(s,a))\right] + \mathbf{E}_{(s,a)\sim\pi_\theta}\left[\log D_\phi(s,a)\right]\end{aligned}$$

- Naively, BC and GAIL cannot outperform the expert, but GAIL can be augmented with additional rewards from the environment:
$$r(s,a) = \lambda_t r_{\text{GAIL}}(s,a) + (1-\lambda_t) r_{test}(s,a)$$
- One can specify $\lambda_t \to 0$ to "warm-start" the agent. or model complex rewards.

Goal Conditioning: Condition rewards based on goal: $V_\theta(s)$ becomes $V_\theta(s,g)$. The learning objective for BC becomes

$$J_{\text{BC}}(\theta,\tau) = \mathbf{E}_{(s_t^j,a_t^j,g^j)\sim\tau}\|a_t^j - \pi_\theta(s_t^j,g^j)\|_2^2.$$

Goal Relabelling [16]: Data augmentation for goal-conditioned reinforcement learning. For a data point of the form $(s_t^j, a_t^j, r_{t+1}^j, s_{t+1}^j, g^j)$ (j is the trajectory label), we can relabel it $(s_t^j, a_t^j, r_{t+1}^j, s_{t+1}^j, g' = s_{t+k}^j)$ (or some other way of sampling the goal).

5.2 Diffusion Models

Stochastic Generative Models: A model that maps a latent distribution $p(\mathbf{z})$ to the data distribution $p(\mathbf{x})$ with conditional distribution $p(\mathbf{x} \mid \mathbf{z})$:

$$p(\mathbf{x} \mid \mathbf{z};\theta) = \mathcal{N}(\mathbf{x} \mid f(\mathbf{z};\theta), \sigma^2 I).$$

- We can maximize the marginal probability of the data:

$$\max_{\theta} p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z}; \theta) p(\mathbf{z}) \, \mathrm{d}\mathbf{z},$$

  but sampling directly yields poor results.

Evidence Lower-Bound (ELBO): Consider a stochastic generative model that learns another parameterized distribution $q_\phi(z \mid x)$ ($\int q_\phi(\mathbf{z} \mid \mathbf{x}) = 1$). Then,

$$\begin{aligned}
\log p(\mathbf{x}) &= \log p(\mathbf{x}) \int q_\phi(\mathbf{z} \mid \mathbf{x}) \, \mathrm{d}\mathbf{z} \\
&= \int q_\phi(\mathbf{z} \mid \mathbf{x}) \log p(\mathbf{x}) \, \mathrm{d}\mathbf{z} \\
&= \mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log p(\mathbf{x}) \right] \\
&= \mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}) \right] + \mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{q_\phi(\mathbf{z} \mid \mathbf{x})}{p(\mathbf{z} \mid \mathbf{x})}) \right] \\
&= \mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}) \right] + \mathcal{D}_{\mathsf{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z} \mid \mathbf{x})) \\
&\geq \mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}) \right] \\
&= \mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log p(\mathbf{x} \mid \mathbf{z}) \right] + \mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log \frac{p(\mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})}) \right] \\
&= \underbrace{\mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log p(\mathbf{x} \mid \mathbf{z}) \right]}_{\text{reconstruction term}} + \underbrace{\mathcal{D}_{\mathsf{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z})))}_{\text{prior matching term}}.
\end{aligned}$$

We wish to learn the intermediate bottlenecked distribution $q_\phi(\mathbf{z} \mid \mathbf{x})$ that **encodes** the data by transforming the input into a distribution over possible latents. The $p_\theta(\mathbf{x} \mid \mathbf{z})$ parameter reverses this by **decoding** the latent vector into its observation. We choose to model the **evidence** as $p(z) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$ and $q_\phi(z \mid x) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \boldsymbol{\sigma}^2_\phi(\mathbf{x})\mathbf{I})$.

Variational Autoencoder (VAE) [17]: A neural network with an encoder ($q_\phi(\mathbf{z} \mid \mathbf{x})$) and a decoder ($p_\theta(\mathbf{x} \mid \mathbf{z})$) that are trained in tandem.

$$p(\mathbf{x}) \xrightarrow[\text{encoder}]{q_\phi(\mathbf{z}|\mathbf{x})} p(\mathbf{z}) \xrightarrow[\text{decoder}]{p_\theta(\mathbf{x}|\mathbf{z})} p_\theta(\mathbf{x} \mid \mathbf{z})$$

Training can be done by feeding a data point $\mathbf{x}$ and predicting $\mu_\phi(\mathbf{x})$ and $\sigma^2_\phi(\mathbf{x})$; then sampling $z \sim \mathcal{N}(\mu_\phi(\mathbf{x}), \sigma^2_\phi(\mathbf{x}))$. Using the sampled z, we compute $\hat{x} = f(\mathbf{z}; \theta)$.

- Notice that the $\mathcal{D}_{\mathsf{KL}}(\cdot \| \cdot)$ term in the loss trains the encoder ϕ, whereas the $\mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\cdot]$ term trains the decoder θ.

Reparameterization Trick: We can transform the loss term

$$\mathbf{E}_{q_\phi(\mathbf{z}|\mathbf{x})} \left[\log p(\mathbf{x} \mid \mathbf{z}) \right] = \mathbf{E}_{\epsilon \sim \mathcal{N}(0,I)} \left[\log p_\theta(\mathbf{x} \mid \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}) \right]$$

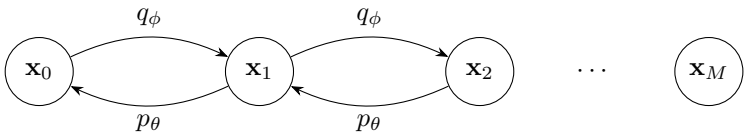
where $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\epsilon}; \mathbf{0}, \mathbf{I})$ is provided as an input. This allows gradients to flow through the sampling step.

Conditional VAEs: VAEs that can generate conditional distributions over data; this can be done by supplying a state number s to the encoder and decoder networks.

Markovian Hiercharchal VAEs (MHVAEs): Wultiple chained VAEs between the data $\mathbf{x}_0$ and the latent vector $\mathbf{x}_T$. The model is Markovian in the sense that each node is a p_θ or q_ϕ transformation of only the previous node:

$$p_\theta(\mathbf{x}_{0:T}) = p_\theta(x_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

and similarly for $q_\phi(\mathbf{x}_{0:T})$.



ELBO for MHVAEs:

$$\begin{aligned}
\log p(\mathbf{x}_0) &= \log \int p_\theta(\mathbf{x}_{0:T}) \, \mathrm{d}\mathbf{x}_{1:T} \\
&= \log \mathbf{E}_{q_\phi(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\frac{p_\theta(\mathbf{x}_{0:T})}{q_\phi(\mathbf{x}_{1:T} \mid \mathbf{z})} \right] \\
&\geq \mathbf{E}_{q_\phi(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q_\phi(\mathbf{x}_{1:T} \mid \mathbf{z})} \right] \\
&= \mathbf{E}_{q_\phi(\mathbf{x}_{1:T}|\mathbf{x}_0} \left[\log \frac{p(\mathbf{z}_T) p_\theta(\mathbf{x} \mid \mathbf{z}_1) \prod_{t=2}^{T} p_\theta(\mathbf{z}_{t-1} \mid \mathbf{z})}{q_\phi(\mathbf{z}_1 \mid \mathbf{x}) \prod_{t=2}^{T} q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1})} \right].
\end{aligned}$$

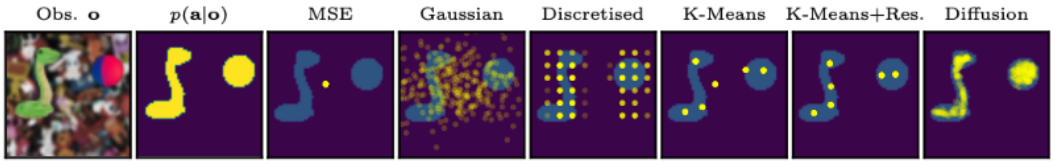


Figure 1: Expressiveness of a variety of models for behaviour cloning in a single-step, arcade claw game with two simultaneous, continuous actions. Existing methods fail to model the full action distribution, $p(\mathbf{a}|\mathbf{o})$, whilst diffusion models excel at covering multimodal & complex distributions.

Figure 1: Figure 1 from *Imitating Human Behaviour with Diffusion Models* by Pearce *et. al.*

Diffusion Models [18]: A generative model that allows for greater expressivity for generating conditional distributions (Figure 1). Diffusion Models are MHVAE where the latent dimension is equal to the data dimension, and encoder is fixed by hyperparameters s.t. the $x_T \sim \mathcal{N}(0, 1)$. Specifically,

$$\begin{aligned}
q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) &= \mathcal{N}(x_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, (1-\alpha_t)I) \\
p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) &= \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I).
\end{aligned}$$

Note that μ_θ now takes in the step t as an input.

- Because the sum of Gaussian distributions is also Gaussian, we can derive that

  $$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\sqrt{\overline{\alpha_t}}\mathbf{x}_0, (1-\overline{\alpha_t})I)$$

  where $\overline{\alpha_t} = \prod_{i=1}^{t} \alpha_i$.

- Note that as $t \to \infty$, we have that $\overline{\alpha_t} \to 0$ so $q(\mathbf{x}_t \mid \mathbf{x}_0) \Rightarrow \mathcal{N}(0, I)$.

ELBO for Diffusion Models:

$$
\begin{aligned}
\log p(\mathbf{x}) &= \log \int p(\mathbf{x}_{0:T}) \, \mathrm{d}\mathbf{x}_{1:T} \\
&= \log \int \frac{p(\mathbf{x}_{0:T}) q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \, \mathrm{d}\mathbf{x}_{1:T} \\
&= \log \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \frac{p(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \\
&\geq \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \log \frac{p(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \\
&= \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \log \frac{p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}{\prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \\
&= \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \log \frac{p(\mathbf{x}_T) p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \prod_{t=1}^{T-1} p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1})}{q(\mathbf{x}_T \mid \mathbf{x}_{T-1}) \prod_{t=1}^{T-1} q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \\
&= \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_T) p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)}{q(\mathbf{x}_T \mid \mathbf{x}_{T-1})} \right] + \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \left[\log \prod_{t=1}^{T-1} \frac{p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1})}{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \right] \\
&= \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \left[\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \right] + \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_T)}{q(\mathbf{x}_T \mid \mathbf{x}_{T-1})} \right] + \sum_{t=1}^{T-1} \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \log \frac{p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1})}{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \\
&= \mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \left[\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \right] + \mathbf{E}_{q(\mathbf{x}_{T-1}, \mathbf{x}_T \mid \mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_T)}{q(\mathbf{x}_T \mid \mathbf{x}_{T-1})} \right] + \sum_{t=1}^{T-1} \mathbf{E}_{q(\mathbf{x}_{t-1}, \mathbf{x}_t, \mathbf{x}_{t+1} \mid \mathbf{x}_0)} \log \frac{p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1})}{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \\
&= \underbrace{\mathbf{E}_{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} \left[\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \right]}_{\text{reconstruction term}} - \underbrace{\mathbf{E}_{q(\mathbf{x}_{T-1} \mid \mathbf{x}_0)} \left[\mathcal{D}_{\mathsf{KL}}(q(\mathbf{x}_T \mid \mathbf{x}_{T-1}) \| p(\mathbf{x}_T)) \right]}_{\text{prior matching term}} \\
&\quad \underbrace{- \sum_{t=1}^{T-1} \mathbf{E}_{q(\mathbf{x}_{t-1}, \mathbf{x}_{t+1} \mid \mathbf{x}_0)} \mathcal{D}_{\mathsf{KL}}(q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \| p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1}))}_{\text{consistency term}}
\end{aligned}
$$

- The **reconstruction term** is the same as the ELBO for VAEs given the first step latent.
- The **prior matching term** has no trainable parameters and ensures that the final latent distribution matches the Gaussian prior (since we assume that $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}; \mathbf{I})$).
- The **consistency term** ensures that the distribution of $\mathbf{x}_t$ obtained from transitioning from noiser samples $\mathbf{x}_{t+1}$ matches the distribution over $\mathbf{x}_t$ obtained from noising cleaner samples $\mathbf{x}_{t-1}$ for all t.

In particular, note that the consistency term depends on both $\mathbf{x}_{t+1}$ and $\mathbf{x}_{t-1}$, which introduces high variance when we try to optimize with Monte-Carlo.

ELBO with Denoising Matching Term: Because diffusion models are a subset of Markovian VAEs, we must have that $q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0)$, where the $\mathbf{x}_0$ term is superfluous due to the Markov property. Expanding with Bayes' rule, we get

$$
q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{q(\mathbf{x}_t \mid \mathbf{x}_0)}.
$$

Substituting into the ELBO derivation, we get a **denoising matching term** of

$$\sum_{t=2}^{T} \mathbf{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \mathcal{D}_{\mathsf{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x})).$$

We can interpret the distirbution $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ as the ground-truth signal because it defines how to denoise a noisy image $\mathbf{x}_t$ with access to the final, completely denoised image $\mathbf{x}_0$.

ELBO by Predicting the Reconstructed Image: We can derive the Gaussian parameterization of $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ (given that the forms of $q(\mathbf{x}_t \mid \mathbf{x}_0)$ and $q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)$ are known):

$$\begin{aligned}
q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) &= \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{q(\mathbf{x}_t \mid \mathbf{x}_0)} \\
&= \frac{\mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, (1-\alpha_t)\mathbf{I})) \mathcal{N}(\mathbf{x}_{t-1}; \sqrt{\overline{\alpha}_{t-1}}\mathbf{x}_0, (1-\overline{\alpha}_{t-1}\mathbf{I}))}{\mathcal{N}(\mathbf{x}_t; \sqrt{\overline{\alpha}_t}\mathbf{x}_0, (1-\overline{\alpha}_t)\mathbf{I})} \\
&\propto \mathcal{N}\left(\mathbf{x}_{t-1}; \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)\mathbf{x}_0}{1-\overline{\alpha}_t}, \frac{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}\mathbf{I}\right) \\
&= \mathcal{N}(\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \boldsymbol{\Sigma}_q(t)).
\end{aligned}$$

We can write $\boldsymbol{\Sigma}_q(t) = \boldsymbol{\sigma}^2 q(t)\mathbf{I}$ where $\boldsymbol{\sigma}_q^2(t) = \frac{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}$. To match this term as well as possible, we parameterize the denoising transition step as a Gaussian. Then, the denoising matching term becomes

$$\begin{aligned}
&\mathcal{D}_{\mathsf{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)) \\
&= \mathcal{D}_{\mathsf{KL}}(\mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q(t)) \| \mathcal{D}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta, \boldsymbol{\Sigma}_q(t))) \\
&= \frac{1}{2}\left[\log \frac{|\boldsymbol{\Sigma}_y|}{|\boldsymbol{\Sigma}_x|} - d + \operatorname{tr}(\boldsymbol{\Sigma}_y^{-1}\boldsymbol{\Sigma}_x) + (\boldsymbol{\mu}_y - \boldsymbol{\mu}_x)^\top \boldsymbol{\Sigma}_y^{-1}(\boldsymbol{\mu}_y - \boldsymbol{\mu}_x)\right] \\
&= \frac{1}{2\boldsymbol{\sigma}_q^2(t)}\left[\|\boldsymbol{\mu}_\theta - \boldsymbol{\mu}_q\|_2^2\right]
\end{aligned}$$

Here, $\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0)$ denotes the noised mean and $\boldsymbol{\mu}_\theta(\mathbf{x}_t)$ denotes the denoised mean. The denoising transition mean can be rewritten as

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)\hat{x}_\theta(\mathbf{x}_t, t)}{1-\overline{\alpha}_t},$$

so the overall consistency term can be written as

$$\operatorname*{argmin}_\theta \mathbf{E}_{t\sim U[2,T]} \mathbf{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\sigma_q^2(t)} \frac{\overline{\alpha}_{t-1}(1-\alpha_t)^2}{(1-\overline{\alpha}_t)^2} \|\hat{\mathbf{x}}(\mathbf{x}_t, t) - \mathbf{x}_0\|_2^2\right].$$

Thus, optimizing a diffusion models is equivalent to learning a neural network to predict the original ground truth image from an arbitrarily noisified version of it.

ELBO by predicting Noise: Recall that

$$\begin{aligned}
q(\mathbf{x}_t \mid \mathbf{x}_0) &= \mathcal{N}\mathbf{x}_t; \sqrt{\overline{\alpha}_t}\mathbf{x}_0, (1-\overline{\alpha}_t)\mathbf{I} \\
\mathbf{x}_t &= \sqrt{\overline{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t}\boldsymbol{\epsilon}_0
\end{aligned}$$

which gives that

$$\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1-\overline{\alpha}_t}\boldsymbol{\epsilon}_0}{\sqrt{\overline{\alpha}_t}}.$$

Then,

$$\begin{aligned}\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0) &= \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)\mathbf{x}_0}{1-\overline{\alpha}_t} \\ &= \frac{\mathbf{x}_t}{\sqrt{\alpha_t}} - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}\sqrt{\alpha_t}}\boldsymbol{\epsilon}_0.\end{aligned}$$

Similarly, the denoising transition mean can be rewritten as

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{\mathbf{x}_t}{\sqrt{\alpha_t}} - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}\sqrt{\alpha_t}}\hat{\epsilon}(\mathbf{x}_t, t).$$

Thus, the consistency term reduces to

$$\operatorname*{argmin}_\theta \mathbf{E}_{t\sim U[2,T]}\mathbf{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\sigma_q^2(t)}\frac{(1-\alpha_t)^2}{(1-\overline{\alpha}_t)\alpha_t}\|\boldsymbol{\epsilon}_0 - \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t)\|_2^2\right],$$

where $\epsilon_0 \sim \mathcal{N}(0, \mathbf{I})$ and $\hat{\epsilon}_\theta$ is a neural network parameterized by θ that predicts the noise level at step t. Training becomes performing gradient decent on

$$\nabla_\theta\|\boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}_\theta(\sqrt{\overline{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t}\boldsymbol{\epsilon}, t)\|_2^2,$$

a standard least-squares objective. Then, sampling can be performed by taking $\mathbf{x}_T, \mathbf{z}_1, \ldots, \mathbf{z}_T \overset{\text{iid}}{\sim} \mathcal{N}(0, \mathbf{I})$ and setting

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t\mathbf{z}$$

Score function: The score function of a distribution is given by $\nabla_\mathbf{x}\log p(\mathbf{x})$. This defines a vector field over the entire data space, and performing gradient ascent on this field leads to data modes.

Tweedie's Formula: For a Gaussian variable $\mathbf{z} \sim \mathcal{N}(\mu_\mathbf{z}, \Sigma_\mathbf{z})$, we have that

$$\mathbf{E}\left[\boldsymbol{\mu}_\mathbf{z} \mid \mathbf{z}\right] = \mathbf{z} + \Sigma_\mathbf{z} + \nabla_\mathbf{z}\log p(\mathbf{z}).$$

ELBO by predicting score: Tweedie's formula enables the prediction of the true posterior mean $\mathbf{x}_t$ given its samples. Given that

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\overline{\alpha}_t}\mathbf{x}_0, (1-\overline{\alpha}_t\mathbf{I})),$$

we have that

$$\mathbf{E}\left[\boldsymbol{\mu}_{\mathbf{x}_t} \mid \mathbf{x}_t \mid =\right] \mathbf{x}_t + (1-\overline{\alpha}_t)\nabla_{\mathbf{x}_t}\log p(\mathbf{x}_t).$$

Since $\boldsymbol{\mu}_{\mathbf{x}_t} = \sqrt{\overline{\alpha}_t}\mathbf{x}_0$, we have that

$$\mathbf{x}_0 = \frac{\mathbf{x}_t + (1-\overline{\alpha}_t)\nabla\log p(\mathbf{x}_t)}{\sqrt{\overline{\alpha}_t}}.$$

We can plug this into the ground-truth denoising transition mean $\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0)$ which yields

$$\begin{aligned}\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0) &= \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)\mathbf{x}_0}{1-\overline{\alpha}_t} \\ &= \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)\frac{\mathbf{x}_t+(1-\overline{\alpha}_t)\nabla\log p(\mathbf{x}_t)}{\sqrt{\overline{\alpha}_t}}}{1-\overline{\alpha}_t} \\ &= \frac{1}{\sqrt{\alpha_t}}\mathbf{x}_t + \frac{1-\alpha_t}{\sqrt{\alpha_t}}\nabla\log p(\mathbf{x}_t).\end{aligned}$$

Then, the approximate denoising transition mean is given by

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\mathbf{x}_t + \frac{1-\alpha_t}{\sqrt{\alpha_t}} s_\theta(\mathbf{x}_t, t).$$

Here, $s_\theta(\mathbf{x}_t, t)$ is a neural network that learns to predict the score function, which is the gradient of $\mathbf{x}_t$ in data space for an arbitrary noise level t. This makes the overall optimization problem reduce to

$$\underset{\theta}{\operatorname{argmin}}\, \mathbf{E}_{t\sim U[2,T]} \mathbf{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left[\frac{1}{2\sigma_q^2(t)} \frac{(1-\alpha_t)^2}{\alpha_t} \|\mathbf{s}_\theta(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)\| \right].$$

We also have that for some $\epsilon_0 \sim \mathcal{N}(0, \mathbf{I})$, that

$$\mathbf{x}_t = \sqrt{\overline{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t}\epsilon_0.$$

Solving for $\mathbf{x}_0$ and equating the two expressions gives a score function of

$$\nabla \log p(\mathbf{x}_t) = -\frac{1}{\sqrt{1-\overline{\alpha}_t}}\epsilon_0.$$

Since this is the direction in the data space to add noise to the image, we move in the opposite direction to increase the log probability.

Conditional Diffusion Models: Conditional diffusion models add arbitrary conditioning information y at each transition step:

$$p(\mathbf{x}_{0:T} \mid y) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t, y).$$

6 Model-Based Reinforcement Learning

Online Planning: Learning strategy where a model of the environment is rolled out to directly predict consequences of actions in order to search for the best actions to take in the current state.

- This is relevant in environments with large state-spaces where computing a good value function is difficult offline. Conditioning on the current state and estimating the value function of the relevant parts of the state space is a more efficient way to allocate resources.

Exhaustive Search: An online planning branching search strategy where the current state of the agent is the root of a search tree. This is infeasible because of the large tree branching factor and large tree depth.

Monte-Carlo Search: A variant of exhaustive search that samples random trajectories. The estimated value of each state is then the average outcome from trajectories starting at that state. Formally, for a root state s, a deterministic transition function T, and a (potentially random) simulation policy π, we simulate k episodes

$$\left\{s, a, R_1^k, S_1^k, R_2^k, S_2^k, A_2^k, \ldots, S_T^k\right\}_{k=1}^{K} \sim T, \pi$$

and evaluate the action value function of the root by mean return:

$$Q(s, a) = \frac{1}{K}\sum_{k=1}^{K} G_k$$

which converges to $q_\pi(s,a)$ as $K \to \infty$. Then, the action in the current state is selected via $a = \text{argmax}_a\, Q(s,a)$.

Upper Confidence Bound (UCB): An alternative to ϵ-greedy for balancing exploration and exploitation. The action selection is weighted by

$$w_t = \underset{a}{\text{argmax}}\, Q_t(a) + c\sqrt{\frac{\log t}{N_t(a)}}$$

where t is the number of times the parent node has been visited, $N_t(a)$ is the number of times the action has been tried.

Monte-Carlo Tree Search (MCTS): We maintain action values Q for the root and nodes internal to a tree expanded through the course of the search. This is used to improve the simulation policy over time. Outside this tree, there are no Q estimates and we fall back on a random policy.

Formally, MCTS consists of a **bandit phase** and a **random phase**. If a node has an unexpanded child, we expand the child (adding it to the tree) and play it out using a random policy. Otherwise, sample a child using UCB and recurse. Each node tracks the number of visits and the average return.

Offline MCTS [19]: Uses MCTS for Q value estimation and action selection at training time. A reactive policy network that imitates the MCTS planner is used at test time. The authors introduces two strategies:

- `UCTtoRegression`: A regression network that regresses to $Q(s,a,w)$ for all actions and selects actions by taking the argument max.
- `UCTtoClassification`: A classifiaction network that, given the same context, directly predicts the best action.

The naive approach of generating $(s, a, Q(s,a))$ tuples is insufficient because the policy state distribution and the MCTS state distribution will diverge. Instead, the **interleaved method** alternates between deploying the learned ϵ-greedy policy to collect trajectories, then uses MCTS as an expert to decide the best action for each state. Then, the learned policy is updated using the optimal actions.

MCTS with Policy/Value Networks: MCTS can be optimized by using a value network to evaluate board positions to prune the tree depth, and a policy networks to select moves and prune the tree breadth.

- Similar to Actor-Critic, these can be trained jointly with a state embedding with separate heads to predict the action and value function.

AlphaGo [20]: First, train cheap policy p_π and expersive policy p_σ using imitation learning on human expert games. Then, train a new policy p_ρ with RL and self-play initialized from the p_σ policy. Train a value network V_θ that predicts the winner of games played by p_ρ against itself. At test time, MCTS is used to select the best move.

The action selection policy of the MCTS is given by

$$a_t = \underset{a}{\text{argmax}}\, Q(s_t,a) + c\sqrt{N(s_t)}\frac{\pi_\theta(a_t \mid s_t)}{1+N(s_t,a_t)}$$

where $\pi_\theta(a \mid s)$ is the prior expert probabilities provided by p_σ, $Q(s_r,a)$ is the average reward collected so far from MC simulations, and $N(s,a)$ is the number of times action a has been taken in state s. A leaf node L is evaluated as

$$V(s_L) = (1-\lambda)V_\theta(s_L) + \lambda z_L$$

where z_L is the roward from the fast rollout p_π (done in parallel) and λ isa mixing parameter. Once the MCTS completes, the algorithm chooses the most visited move from the root position.

AlphaGoZero [21]: AlphaGo with "zero" human examples to seed the policy, due to having a better training loop to discover stronger actions in training using offline MCTS. The UCB during training is the same as the base AlphaGo, except the policy $\pi_\theta(a_t \mid s_t)$ is changing during the policy. Once MCTS completes, the algorithm trains the policy π_θ against the visit distribution $\hat{\pi}$:

$$\hat{\pi}(a) = \frac{n(a)}{\sum_{a'} n(a')}.$$

Then, the next action for the self-play loop is sampled from $\hat{\pi}$ to encourage exploration. The value function is simply regressed to the outcome of the game.

- Motivation: From Hamrick *et. al.* [22], planning is more important at training time because it leads to more intelligent exploration.
- Value function approximations are used for leaf nodes (no full rollouts). Value functions are only updated when the game terminates. This techinque is called the **terminal value function**.
- Note that the neural network has a single backbone that accepts a state input, and two heads that predict the action distribution and the value function.
- Similarly to AlphaGo, MCTS is still used a test-time.

7 Model-Free Reinforcement Learning

Model-Free Reinforcement Learning: Policy interacts with a learned model of the environement (dynamics $p(s_{t+1} \mid s_t, a_t)$ and $r(s_t, a_t)$ are simulated/learned). This can be done using neural networks, Gaussian processes, etc.

- Models are useful for lookahead, such as MCTS, or used for synthetic experience generation that augments real experience to boost a model-free algorithm such as PPO.
- Goal is to have high rewards with little interactions with the actual environment.

MBRL with Learned Models: We first run the policy and update experience tuples dataset $\mathcal{D}_{\text{env}}$. Then, the dynamics model is regressed to $\mathcal{D}_{\text{env}}$:

$$\min_f \mathbf{E}_{s_t, a_t, s_{t+1} \sim \mathcal{D}_{\text{env}}} \|s_{t+1} - f(s_t, a_t)\|.$$

Then the policy is optimized using rollouts from the learned function f.

- State distribution mismatch is a big problem here because any mismatch between f and the ground truth will be exploited by the policy.
- In practice, the difference between the current state and next state is modelled: $s' = s + f_\phi(s, a)$
- The reward in this case is defined as the distance to some final state s^*.

Model-predictive Control (MPC): An instance of MBRL with Learned Models where an action is executed sequentially, the following state is observed, and the model replans for future steps. This mitigates the problem of error accumulation through unrolling.

MBRL in Low-Dimensional States: The state is observed and given in low-dimensional space (e.g., object locations and robot configurations).

MBRL from High-Dimensional Sensory Input: The state is observed and given in terms of high-dimensional sensory input such as video. The two approaches to addressing this is recurrence on image space (e.g., deterministc video prediction) or recurrence in latent space.

7.1 MBRL in State-Space

Probabilistic Ensembles with Trajectory Sampling (PETS) [23]: Model representations of the environment are augmented with uncertainty predictions. Specifically, there are two types: **epistemic uncertainty** is due to the lack of data (e.g., for a deterministic system), and **aleatoric uncertainty** is due to the inherent stochasticity of the system The model outputs a Gaussian distribution over next states given the current state and action by predicting the mean and covariance matrix:

$$\mathcal{L}_\phi = -\frac{1}{N}\sum_{i=1}^{N} \log p_\phi(s_i' \mid s_i, a_i).$$

An ensemble of networks (3 networks) is used to make this prediction in parallel; the variance of the results across the ensemble estimates the epistemic uncertainty. The reward of an action sequence is done by averaging across **particles**, where each particle is a separate network.

- Aleatoric uncertainty implies that standard regressive predictions will not perform well and motivates the use of architectures capable of handling multi-modality such as diffusion networks.
- Use $\ln\text{-}\exp$ trick to predict positive values for variances.

The final algorithm is: Use the cross-entropy method (an evolutionary search algorithm) to optimize over action sequences $a^*_{t,\ldots,T}$ by unrolling the model ensemble and reward averaging, executing the first action a^*_t and updating the environment dataset $\mathcal{D}_{\text{env}}$ with tuple (s_t, a^*_t, s_{t+1}).

- Suppose there are B models in the ensemble. In CEM, we first sample an action sequence $\vec{a}_{t\ldots T}$ from the learned action sequence distribution. For each model $b \in \{1, \ldots, B\}$, set $s_t^{(b)} = s_t$ and compute for each following step,

$$\begin{aligned} s_{k+1}^{(b)} &\sim p_k(s_k^{(b)}, a_k) \\ r_k^{(b)} &= r(s_k^{(b)}, a_k) \\ R^{(b)}(\vec{a}) &= \sum_{k=t}^{T} r(s_k^{(b)}, a_k) \end{aligned}$$

  Then, the return for $\vec{a}$ is said to be the average across all $R^{(b)}(\vec{a})$. Note that the states are not averaged across ensembles.

7.2 MBRL in Sensory-input Space

Approaches to MBRL in Sensory-input Space: There are two main approaches to MBRL in sensory-input space:

- **Unrolling in the observation space**: The environment model is given observations $o_{t-k}, \dots, o_t, a_t$ and asked to predict o_{t+1}.
- **Unrolling in the latent space**: The environment model is given a latent state s_t representing observations $o_{t-k}, \dots, o_t$ as well as a_t and asked to predict s_{t+1}.

Action-Conditional Video Predicting using Deep Networks in Atari Games [24]: Trains a neural network that given an image sequence and an actions, predicts the pixels of the next frame using a standard L_2 objective. Using a naive approach, we face the problem of **error accumulation** where test-time rollouts have a distribution shift (similar to imitation learning). This can be solved by:

(1) Progressively unrolling the horizon k at training time so that the model learns to handle its mistakes:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|f(a_{t+1}^{(i)}, f(a_t^{(i)}, o_t^{(i)}; \theta); \theta) - o_{t+2}^{(i)}\| + \|f(a_t^{(i)}, o_t^{(i)}; \theta) - o_{t+1}^{(i)}\|.$$

This fails for non-regressive, multi-modal models and highly stochastic environments because the model rollout and the ground truth and deviate significantly.

(2) Add noise to each observation to simulate predictions of the networks.

Model-Based Reinforcement Learning for Atari [25]: Similar architecture to before but also make predictions on rewards: First initialize π_θ and $\mathcal{D}_{\text{env}}$. In each iteration, run the policy and update $\mathcal{D}_{\text{env}}$. Then, train the dynamics model by regressing $f(\{o_{t-4}, \dots, o_t\}, a_t, z; \theta)$ to (o_{t+1}, r_{t+1}). Then, sample $o_{t-4}, \dots, o_t \sim \mathcal{D}_{\text{env}}$ and collect simulated experience $\mathcal{D}_{\text{model}}$ for 50 time steps using π_θ to select actions. Finally, update π_θ using PPO on trajectories $\mathcal{D}_{\text{model}} \cup \mathcal{D}_{\text{env}}$.

- The paper uses a multi-model VQ-VAE for $f(\cdot; \theta)$. This inherently operates similar to an ensemble by encorporating epistemic uncertainty, which mitigates the need to use an actual ensemble (as used in PETS).
- Compared to a base model-free PPO model, the model-based method required much less (actual) environment interactions to read human-level performance.
- Disadvantages: video prediction loss does not reflect task performance (predicts task-irrelevant objects), and is extremely slow.

Temporal Difference for Model Predictive Control (TD-MPC) [26]: Train an encoder E_ϕ and a decoder D_ψ with an autoencoder loss

$$\min_{\phi,\psi} \|D_\psi(E_\phi(o_t)) - o_t\|.$$

The encoder also encoporates task-relevant losses such as action prediction, reward prediction, and Q-value prediction. Then, the dynamics prediction is performed entirely on the latent space:

$$\min_\theta \|f_\theta(E_\phi(o_{t-K}), \dots, E_\phi(o_t)) - E_\phi(o_{t+1})\|.$$

Tasked-Oriented Latent Dynamics (TOLD): TOLD minimizes the objective

$$\mathcal{J}(\theta; \Gamma) = \sum_{i=t}^{t+H} \omega^{i-t} \mathcal{L}(\theta; \Gamma_i)$$

where

$$\mathcal{L}(\theta;\Gamma_i) = \lambda_1 \underbrace{\|R_\theta(\mathbf{z}_i,\vec{a}_i) - r_i\|_2^2}_{\text{reward}} + \lambda_2 \underbrace{\|Q_\theta(\mathbf{z}_i,\vec{a}_i) - (r_i + \gamma Q_{\theta^-}(\mathbf{z}_{i+1},\pi_\theta(\mathbf{z}_{i+1})))\|_2^2}_{\text{value}} + \lambda_3 \underbrace{\|d_\theta(\mathbf{z}_i,\vec{a}_i) - h_{\theta^-}(\vec{s}_{i+1}))\|_2^2}_{\text{latent state consistency}}.$$

Here, θ^- represent the parameters of a target network that is a moving average of θ. Then, the policy minimizes

$$\mathcal{J}_\pi(\theta;\Gamma) = -\sum_{i=t}^{t+H} \omega^{i-t} Q_\theta(\mathbf{z}_i, \pi_\theta(\mathrm{sg}(\mathbf{z}_i))),$$

where the sg operator represents stop-gradient, preventing the gradient from updating the latent state encoder.

Planning in Latent-Space: We can estimate the return of a trajectory via

$$\mathbf{E}_\Gamma \underbrace{\gamma^H Q_\theta(\mathbf{z}_H,\vec{a}_H)}_{\text{value}} + \underbrace{\sum_{t=0}^{H-1} \gamma^t R_\theta(\mathbf{z}_t,\vec{a}_t)}_{\text{rewards}}.$$

This is used to maximize an evolutionary search method such as CEM (similar to PETS, but with a different dynamics model and a terminal value function).

Planning with a Learned Model (MuZero) [27]: For a given game state encodes each observation with a learned encoder $h_\theta(o_1,\ldots,o_t) = s^{(0)}$. Then, a prediction network predicts a distribution over actions and a state value $f_\theta(s^{(k)}) = (\vec{p}^{(k)}, \nu^{(k)})$. Finally, the learned dynamics function gives $g_\theta(s^{(k-1)}, a^{(k)}) = (r^{(k)}, s^{(k)})$.

Doing this process in a tree search similar to AlphaZero gives an improved policy (action distribution by normalizing visitation counts) and an improved value function (average Qs over actions). Note that the tree search is executed with the actual environment, not the learned simulator.

- The learned model update is by regressing the state embedding to match the policy, reward, and n-step value bootstrap:

  $$v_t \approx u_{t+1} + \gamma u_{t+2} + \cdots + \gamma^{n-1} u_{t+n} + \gamma^n \nu_{t+n},$$

  where the last term is the improved state value estimate the MCTS. Note that a decoding is not required or helpful. A consistency loss can also be added.

MuZero Reanalyze [28]: We can reuse old trajectories by again training policy and reward to its corresponding values $p_t \to \pi_t^{T'}$, $r_t \to u_t^T$. Instead of n-step bootstrapping, because the trajectory is now modified, we just directly update with MCTS $v_t \to \nu_t^{T''}$, where T represents the trajectory in the ground-truth environment, T' is the original sampled trajectory in the latent state, and T'' is the reused trajectory generated by post-training with MCTS.

- This can increase replay by $10\times$, using only 10% new trajectories with 90% reanalyzed old trajectories.
- Only offers a significant advantage over MuZero in strategy games where planning in relevant (e.g. Chess, Go, but not Pacman).

7.3 MBRL with Guided Diffusion

Diffuser [29]: Each trajectory τ is formulated as a temporal image where one axis is time and the other is the state-action feature dimension.

$$\boldsymbol{\tau} = \begin{pmatrix} s0 & s1 & \cdots & s_H \\ a_0 & a_1 & \cdots & a_{H-1} \end{pmatrix} \in \mathbb{R}^{H \times d}$$

where $d = d_s + d_a$ is the dimension of the concactenated feature and action space. The forward noising process is a standard DDPM applied to entire trajectories:

$$q(\tau_k \mid \boldsymbol{\tau}_{k-1}) = \mathcal{N}(\boldsymbol{\tau}_k; \sqrt{\alpha_k}\boldsymbol{\tau}_{k-1}, (1-\alpha_k)\mathbf{I}).$$

The loss is given by

$$\mathcal{L}(\theta) = \mathbf{E}_{\boldsymbol{\tau}_0 \sim \mathcal{D}, \boldsymbol{\epsilon} \sim \mathcal{N}(0,\mathbf{I})} \|\boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}_\theta(\sqrt{\overline{\alpha}_k}\boldsymbol{\tau}_0 + \sqrt{1-\overline{\alpha}_k}\boldsymbol{\epsilon}, k)\|.$$

In practice, $\hat{\boldsymbol{\epsilon}}_\theta$ is implemented as a 1-D convolutional neural network to preserve temporal invariance.

For plannin,g a guidance function $J(\tau)$ is used to model predicted return, negative cost, goal satisfaction, or constraint satisfaction. We have that

$$\tilde{p}_\theta(\boldsymbol{\tau}) \propto p_\theta(\boldsymbol{\tau}) h(\boldsymbol{\tau}),$$

where $h(\boldsymbol{\tau}) = exp(\eta J(\boldsymbol{\tau}))$. Taking the log-gradient, this yields

$$\nabla_{\boldsymbol{\tau}} \log \tilde{p}(\boldsymbol{\tau}) = \nabla_{\boldsymbol{\tau}} \log p_\theta(\tau) + \eta \nabla_{\boldsymbol{\tau}} J(\boldsymbol{\tau}).$$

Here, p_θ supplies the trajectory prior, while the guide supplies reward- or constrained-directed gradients. Accounting for the constraints, the diffusion sampler becomes

$$\boldsymbol{\tau}_{k-1} \sim \mathcal{N}(\boldsymbol{\tau}_{k-1}; \boldsymbol{\mu}_\theta(\boldsymbol{\tau}_k, k) + \lambda \boldsymbol{\Sigma}_k \nabla_{\boldsymbol{\tau}} J(\boldsymbol{\tau}), \boldsymbol{\Sigma}_k).$$

We combine this with goal in-painting to implement goal-conditioned planning. The generation process is given by

$$\begin{aligned} \mathbf{x}_{0|t} &= \frac{1}{\sqrt{\overline{\alpha}_t}} \left(\mathbf{x}_t - \sqrt{1-\overline{\alpha}_t} \epsilon_\theta(\mathbf{x}_t, t) \right) \\ \mathbf{x}_{0|t}^{\text{guided}} &= \mathbf{x}_{0|t} + c_t \nabla_{\mathbf{x}_{0|t}} r(\mathbf{x}_{0|t}) \\ \mathbf{x}_{t-1} &= \sqrt{\overline{\alpha}_{t-1}} \mathbf{x}_{0|t}^{\text{guided}} + \sqrt{1-\overline{\alpha}_{t-1}-\sigma_t^2} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) + \boldsymbol{\sigma}_t \boldsymbol{\epsilon} \end{aligned}$$

Note that the roward function r here needs to be both diffentiable and trained on both unnoised/noised trajectories.

Conditional Diffusion Generation: Suppose we then try to condition the generation process on some data y. We have that

$$\begin{aligned} \nabla \log p(\mathbf{x}_t \mid y) &= \nabla \log \frac{p(\mathbf{x}_t) p(y \mid \mathbf{x}_t)}{p(y)} \\ &= \nabla \log p(\mathbf{x}_t) + \nabla \log p(y \mid \mathbf{x}_t) - \nabla \log p(y). \end{aligned}$$

Taking the argmax eliminates the last term because it does not depend on the data.

- The distribution $p(y \mid \mathbf{x}_t)$ is a simple classifier. Note that this needs to be trained on both clean and noised images.

- The two terms can be weighted as follows:

$$\nabla_{\mathbf{x}} \log p_\gamma(\mathbf{x} \mid y) = \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \gamma \nabla_{\mathbf{x}} \log p(y \mid \mathbf{x}).$$

Goal Planning through Inpainting: Suppose the guidance function specifies only the initial and goal states. Then, a straightforward method is to substitute $s_0^{(t)}$ and $s_n^{(t)}$ with the initial and goal states, respectively, at each denoising time step. Formally, if M is a binary mask over the trajectory array where $M = 1$ on constrained entries and 0 elsewhere, then

$$\boldsymbol{\tau}_k := M \odot \boldsymbol{\tau}_{\text{cond}} + (1 - M) \odot \boldsymbol{\tau}_k.$$

Diffusion-ES [30]: Standard ES requires a mutation operator, which is typically of the form $x' = x + \sigma\boldsymbol{\epsilon}$, which likely deviates from the manifold of valid trajectories. Diffusion-ES instead partially noises the trajectories via

$$\overline{x} = \sqrt{\overline{\alpha}_{n_k}}\mathbf{x} + \sqrt{1 - \overline{\alpha}_{n_k}}\boldsymbol{\epsilon}$$

for $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$ and then denoises via $x' \sim p_\theta(x \mid \overline{x})$.

8 Offline Reinforcement Learning

Offline/Batch Reinforcement Learning: Methods that learn from a fixed experience buffer $\mathcal{D} = \{(s_i, a_i, s_i', r_i)\}_{i=1}^k$ of state-action-reward trajectories sampled from some unknown policy π_β. Note that the trajectories in the buffer may have high rewards or low rewards. The goal is to compute a policy π_θ s.t.

$$\max_\theta \mathbf{E}_{\tau \sim \pi_\theta} \sum_t r(s_t, a_t)$$

- Offline RL methods should be able to beat a behavior cloning baseline because it has access to the rewards associated with each state-action pair. This allows it to **stitch** optimal behaviors.
- Offline RL is useful for incorporating existing, static datasets from non-expert policies to problems where the environment interactions are unsafe or expensive.

Offline RL with Actor-Critic: One approach is to apply a standard off-policy RL algorithm like actor-critic (e.g., DDPG) to an offline dataset. This corresponds to critic updates

$$\min_\phi \mathbf{E}_{(s,a,a') \sim \mathcal{D}} \left(Q_\phi^\pi(s,a) - \left(r(s,a) + \gamma \mathbf{E}_{a' \sim \pi_\theta(\cdot \mid s')} Q_\phi^\pi(s', a') \right) \right)^2$$

and actor updates

$$\max_\theta \mathbf{E}_{s \sim \mathcal{D}, a \sim \pi_\theta(\cdot \mid s)} Q_\phi^\pi(s,a) \nabla_\theta J(\theta) = \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(a_i^\pi \mid s_i) Q_\phi^{\pi_\theta}(s_i, a_i^\pi)$$

where $a_i^\pi \sim \pi_\theta(\cdot \mid s_i)$.

- Off-policy deep reinforcement learning fails when truly off-policy, even when the algorithm and base dataset are the same. This is due to **distribution shift** problem: a' is sampled from the policy and out of distribution from the dataset. This leads to overestimation on OOD actions.
- Specifically, suppose we train a reward model $\hat{r}_\theta(x, a)$ that approximates $r(x, a)$ in expectation under $p_{\text{data}}(a, x)$, but is incorrect on unseen actions. The policy then exploits errors in this reward function.

Filtered Behavior Cloning: A simple baseline is to filter the dataset by taking only top trajectories by reward $\tilde{\mathcal{D}} = \{\tau \mid r(\tau) > \eta\}$ and train only on those state-action pairs.

Combining Behavior Cloning with TD3 [31]: We add a behavior cloning term to the TD3 loss to keep predicted actions close to the given actions:

$$J(\theta) = \mathbf{E}_{(s,a)\sim\mathcal{D}}\left[-\lambda Q_\phi^{\pi_\theta}(s, \pi_\theta(s)) + \|\pi_\theta(s) - a\|^2\right]$$

with parameters $\lambda = \alpha/\mathbf{E}_{\mathcal{D}}\left[|Q(s,a)|\right]$ and $\alpha = 2.5$. The Q_ϕ function is learned with TD(0).

Batch-Constrained Q Learning (BCQ) [32]: Constrain actions to only actions in the dataset, and only consider actions that are likely under the data distribution. This can be done with a generative model such as a conditional VAE or a diffusion model $G_\omega(a \mid s)$. Then, sample an action $a_i' \sim G_\omega(s)$ and take the action $a_i = a_i' + \xi_\phi(s,a)$ where ξ_ϕ adds a small perturbation near the dataset. The action taken is given by

$$a^* = \operatorname*{argmax}_i Q(s, a_i).$$

Note that $G_\omega(s)$ does not use rewards from $\mathcal{D}$. The ξ_ϕ network may be implemented as

$$\xi_\phi(s,a) = \Phi \tanh(f_\phi(s,a))$$

and trained to maximize

$$\mathbf{E}_{s\sim\mathcal{D}, a\sim G_\omega(\cdot|r)} Q_{\theta_1}(s, a + \xi_\phi(s,a)).$$

The Bellman backup for updating Q_θ is similar to that of the TD3:

$$\begin{aligned}
y &= r + \gamma \max_{a_i}\left[\lambda \min_{j\in\{1,2\}} Q_{\overline{\theta}_j}(s', a_i)\right] \\
\theta_j &:= \operatorname*{argmin}_{\theta_j} \sum \left(Q_{\theta_j}(s,a) - y\right)^2 \\
\phi &:= \operatorname*{argmax}_{\phi} \sum Q_{\theta_1}(s, a + \xi_\phi(s,a))
\end{aligned}$$

where $a \sim G_\omega(s)$. Then, the target networks are soft-updated.

Conservative Q Learning [33]: In conservative models, the estimated reward model $\hat{r}_\theta(x,a)$ is "conservative" in the sense that $\hat{r}_\theta(x,\overline{a}) \le r(x,\overline{a})$ on unseen actions. This loss is given by

$$\begin{aligned}
\min_\phi \max_\mu \quad & \mathbf{E}_{(s,a,s')\sim\mathcal{D}}\left(Q_\phi^\pi(s,a) - \left(r(s,a) + \gamma \mathbf{E}_{a'\sim\pi_\theta(\cdot|s')} Q_\phi^\pi(s',a')\right)\right)^2 \\
& + \alpha \mathbf{E}_{s\sim\mathcal{D}, a\sim\mu(\cdot|s)} Q_\phi^\pi(s,a) - \alpha \mathbf{E}_{(s,a)\sim\mathcal{D}} Q_\phi^\pi(s,a) + \mathbf{E}_{s\sim\mathcal{D}} \mathcal{H}(\mu(\cdot \mid s)),
\end{aligned}$$

where μ is an auxiliary action distribution to probe where the critic is assigning large values. The terms of the loss correspond to the standard TD learning critic update, increased loss for large Q values, decreased loss for Q values for in-dataset actions, and an regularization term to prevent μ from collapsing into a single argmax action.

- Consider finding

  $$\max_\mu \mathbf{E}_{a\sim\mu(\cdot|s)} Q(s,a) + \mathcal{H}(\mu(\cdot \mid s))$$

  for a fixed state s. The **partition function** is defined as

  $$Z(s) = \sum_{\overline{a}} \exp Q(s, \overline{a})$$

Then, the Boltzmann/softmax form of the optimal distribution is expressed as

$$\mu^*(a \mid s) = \frac{\exp Q(s,a)}{Z(s)}.$$

taking the log and rearranging yields

$$Q(s,a) = \log \mu(a \mid s) + \log Z(s)$$
$$\mathbf{E}_{a\sim\mu(\cdot|s)} Q(s,a) = \mathbf{E}_{a\sim\mu(\cdot|s)} \log \mu(a \mid s) + \log Z(s).$$

Since the entropy term is exactly $\mathbf{E}_{a\sim\mu(\cdot|s)} \log \mu(a \mid s)$, the optimal distribution is simply

$$Z(s) = \log \sum_a \exp Q(s,a)$$

The new expression becomes the **conservative value-learning objective**:

$$\min_\phi \mathbf{E}_{(s,a,s')\sim\mathcal{D}} \left(Q^\pi_\phi(s,a) - \left(r(s,a) + \gamma \mathbf{E}_{a'\sim\pi_\theta(\cdot|s')} Q^\pi_\phi(s',a')\right)\right)^2$$
$$- \alpha \mathbf{E}_{(s,a)\sim\mathcal{D}} Q^\pi_\phi(s,a) + \alpha \log \sum_{\overline{a}} \exp Q^\pi_\phi(s,\overline{a}).$$

The corresponding actor updates are simply given by

$$\max_\theta \mathbf{E}_{s\sim\mathcal{D}, a\sim\pi_\theta(\cdot|s)} Q^\pi(s,a)$$

Advantage-Weighted Behavior Cloning: A simple extension of behavior cloning to offline RL is to weight the contribution of each state-action-reward pair by their corresponding advantage:

$$\operatorname*{argmax}_\theta \mathbf{E}_{(s,a)\sim\mathcal{D}} \left[\log \pi_\theta(a \mid s) \exp A(s,a)\right].$$

This requires computing advantages $A(s,a) = Q(s,a) - V(s)$ without querying OOD actions.

Advantage-Weighted Regression (AWR) [34]: Suppose the buffer is generated with some policy π_β. We can estimate the state-value function with Monte Carlo:

$$V^{\pi_\beta}(s) := \min_\phi \mathbf{E}_{s_t\sim\mathcal{D}} \|V^{\pi_\beta}_\phi(s_t) - \sum_{t'=t}^{T} r(s_{t'}, a_{t'})\|^2$$
$$A^{\pi_\beta}(s_t, a_t) := \sum_{t'=t}^{T} r(s_{t'}, a_{t'}) - V^{\pi_\beta}_\phi(s_t) = G_t - V^{\pi_\beta}_\phi(s_t)$$

Then, we extract the policy

$$\hat{\theta} := \operatorname*{argmax}_\theta \mathbf{E}_{(s,a)\sim\mathcal{D}} \left[\log \pi_\theta(a \mid s) \exp\left(\frac{1}{\beta} A^{\pi_\beta}(s,a)\right)\right].$$

Here, β is a parameter that controls the shapness of reweighting. AWR is useful because it avoids maximizing $Q(s,a)$ over arbitrary actions, and instead only fits the policy to actions that actually occurred in the dataset.

- This can be adapted to be an online off-policy algorithm by alternating between updating π_θ and augmenting the dataset via $\mathcal{D} := \mathcal{D} \cup \{\tau_i\}$ sampled from π_θ.

- Standard MC or TD learning for the Q or V functions reflect a mixture of policies of variable competence, when we want to extract the policy with the best performance.

Boltzmann Reweighting of Policy: We show that the AWR algorithm is exactly the KL-regularized policy improvement. Let π and μ be two policies. From the performance difference lemma,

$$J(\pi) - J(\mu) = \mathbf{E}_{\tau \sim p_\pi} \sum_{t=0}^{\infty} \gamma^t A^\mu(s_t, a_t) = \mathbf{E}_{s \sim d^\pi, a \sim \pi(\cdot|s)} A^\mu(s, a) \approx \mathbf{E}_{s \sim d^\mu, a \sim \pi(\cdot|s)} A^\mu(s, a),$$

where we assume that $\mathbf{E}_{s \sim d^\mu} \left[D_{\mathsf{KL}}(\pi \| \mu) \right] \leq \epsilon$ for the approximation to be valid. The Lagrangian is given by

$$\begin{aligned}
\mathcal{L}(\pi, \beta) &= \int_s d_\mu(s) \int_a \pi(a \mid s) A^\mu(s, a) \, \mathrm{d}a \, \mathrm{d}s - \beta \left(\int_s d_\mu(s) \mathcal{D}_{\mathsf{KL}} \left(\pi(\cdot \mid s) \| \mu(\cdot \mid s) \right) \, \mathrm{d}s - \epsilon \right) \\
&= \int_s d_\mu(s) \int_a \pi(a \mid s) A^\mu(s, a) \, \mathrm{d}a \, \mathrm{d}s - \beta \left(\int_s d_\mu(s) \int_a \pi(a \mid s) \log \frac{\pi(a \mid s)}{\mu(a \mid s)} \, \mathrm{d}a \, \mathrm{d}s - \epsilon \right)
\end{aligned}$$

Taking the functional derivative, we get that

$$\frac{\delta \mathcal{L}}{\delta \pi(a \mid s)} = d_\mu(s) A^\mu(s, a) - \beta \, d_\mu(s) \left(\log \frac{\pi(a \mid s)}{\mu(a \mid s)} + 1 \right)$$

Setting for zero and solving for π, we get

$$d_\mu(s) A^\mu(s, a) - \beta \, d_\mu(s) \left(\log \frac{\pi(a \mid s)}{\mu(a \mid s)} + 1 \right) = 0$$

$$\frac{\pi(a \mid s)}{\mu(a \mid s)} = \exp \left(\frac{1}{\beta} A^\mu(s, a) - 1 \right) = e^{-1} \cdot \exp \left(\frac{1}{\beta} A^\mu(s, a) \right)$$

Absorb constants into normalization:

$$\pi^*(a \mid s) \propto \mu(a \mid s) \exp \left(\frac{1}{\beta} A^\mu(s, a) \right)$$

Then, project π^* onto the manifold of parameterized policies:

$$\begin{aligned}
&\operatorname*{argmin}_\pi \; \mathbf{E}_{s \sim \mathcal{D}} \left[D_{\mathsf{KL}} \left(\pi^*(\cdot \mid s) \| \pi(\cdot \mid s) \right) \right] \\
&= \operatorname*{argmax}_\pi \; \mathbf{E}_{s \sim d_\mu(s), \, a \sim \mu(\cdot|s)} \left[\log \pi(a \mid s) \exp \left(\frac{1}{\beta} A^\mu \right) \right].
\end{aligned}$$

Expectile Regression: The τ-expectile is the mean with asymmetric weighting

$$\operatorname*{argmin}_{m_\tau} \mathbf{E}_{x \sim X} L_2^\tau (x - m_\tau)$$

where $L_2^\tau(u)$ is τu^2 for $u \geq 0$ and $(1 - \tau) u^2$ for $u < 0$. The mean is equivalent to the 0.5-expectile. This is useful in offline RL because at a fixed state s, several dataset actions with different $Q(s, a)$ values may exist. The upper expectile of these values acts like a soft approximation to the best in-distribution action value.

Implicit Q Learning (IQL) [35]: IQL avoids querying OOD actions entirely by learning a state-value function $V(s)$, approximated as the upper expectile of $Q(s, a)$ for $a \sim \mathcal{D}(\cdot \mid s)$. We modify TD learning to use L_2^τ:

$$\begin{aligned}
V_\psi(s) &:= \operatorname*{argmin}_V \mathbf{E}_{(s,a) \sim \mathcal{D}} L_2^\lambda (V(s) - Q_\phi^{\mathcal{D}}(s, a)) \\
Q_\phi^{\mathcal{D}}(s, a) &:= \operatorname*{argmin}_Q \mathbf{E}_{(s,a,s') \sim \mathcal{D}} \left(Q(s, a) - (r(s, a) + \gamma V_\psi(s')) \right)^2.
\end{aligned}$$

The policy is extracted via AWR:

$$\underset{\pi}{\operatorname{argmax}}\, \mathbf{E}_{(s,a)\sim\mathcal{D}}\left[\log \pi(a \mid s) \exp\left(\frac{1}{\beta}\left(Q_\phi^{\mathcal{D}}(s,a) - V_\psi(s)\right)\right)\right].$$

9 Exploration

ϵ-greedy Disadvantages: The probability of identifying a favorable trajectory decreases as the number of actions and the time-horizon until the reward increases.

Motivation for Exploration: Motivation can be **extrinsic**, or driven by some external reward, or **intrinsic**, being moved to do something because it is inherently enjoyable (e.g., curiosity, exploration, novelty, etc.). The latter is important because it is task independent and free of human supervision.

- Methods for inducing curiosity-driven exploration include: using ensembles of Q functions to model uncertainty, **state counting** to approximate distributions of sampled states, measuring **model prediction error**, and measuring **reachability** from already explored states.

9.1 Bandits

Multi-armed Bandit Problem: At each time step t, an agent chooses arm $k \in [K]$ and plays it, yielding reward $r_{k,t} \sim \mathcal{P}_k$ where $\mathbf{E}[\mathcal{P}_k] = \mu_k$. The distributions $\mathcal{P}_k$ and means μ_k are unknown to the agent. The objective is to maximize the cumulative rewards, over a finite or infinite horizon. This is the simplest example of the **exploration-exploitation tradeoff**.

Action-Value: The action-value for an action a is its mean reward

$$q_*(a) = \mathbf{E}[r_t \mid a_t = a].$$

- Suppose the agent forms estimates $\forall a \in \mathcal{A}, Q_t(a) \approx q_*(a)$. Define the **greedy action** at time t as

  $$a_t^* = \underset{a}{\operatorname{argmax}}\, Q_t(a).$$

  Then, if $a_t = a_t^*$, the agent is **exploiting**; otherwise, the agent is **exploring**.

Regret: Let $\mu^* = \max_{a\in\mathcal{A}} \mu_a$ be the maximum expected mean reward. The cummulative regret up to time t is defined as

$$I_t = \sum_{i=1}^{t} \Delta_{a_i} = \sum_{i=1}^{t} (\mu^* - \mu_{a_t}).$$

Equivalently, we have that

$$\mathbf{E}[I_t] = \sum_{a\in\mathcal{A}} \Delta_a \mathbf{E}[N_a(t)].$$

Action-Value Estimates: We focus on one action for simplicity. Suppose the rewards for the first $n-1$

timesteps are $r_1, \ldots, r_{n-1}$. Then, the estimate is simply the mean:

$$Q_n = \frac{1}{n-1}(r_1 + \cdots + r_{n-1}).$$

The online update rule is given by

$$Q_{n+1} := Q_n + \frac{1}{n}(r_n - Q_n).$$

Fixed Exploration Action Selection: We allocate a fixed time period to explore, trying bandits uniformly at random. Then, mean rewards are formed for all actions:

$$Q_t(a) = \frac{1}{N_t(a)} \sum_{i=1}^{t-1} r_i \mathbf{1}(a_i = a)$$

(assuming that each bandit is a Bernoulli r.v.). Then, the action selected at each time step is the maximum among all estimated mean rewards. In an infinite horizon setting, the fixed exploration strategy will always exploit.

ϵ-Greedy Action Selection: In ϵ-greedy, the agent explores with probability ϵ and explores otherwise. This is the simplest way to balance exploration and exploitation. Constant ϵ produces linear total regret:

$$I_t \geq \frac{\epsilon}{K} \sum_{a \in \mathcal{A}} \Delta_a,$$

implying that I_t/t approaches a constant as $t \to \infty$.

9.2 Bayesian Exploration

Bayesian Bandits: Bayesian bandits compute the posterior distribution of rewards

$$p\left[\mathcal{R} \mid a_1, r_1, \ldots, a_{t-1}, r_{t-1}\right].$$

This is computed by specifying a prior $p(\mathcal{R})$ and applying Bayes' rule. In other words, instead of only maintaining a point estimate $Q_t(a)$, Bayesian bandits maintain a belief distribution over the unknown mean reward, allowing the agent to distinguish between two arms with the same empirical mean but different uncertainty.

Thompson Sampling: At each time step, sample from the mean reward distributions:

$$\forall k \in [K], \overline{\theta}_k \sim \hat{p}(\theta_k).$$

Then, action $a_t = \operatorname{argmax}_a \overline{\theta}_a$ is chosen. This sampling scheme enables arms with lower expected mean value but high variance to be explored. Then, the mean reward posterior distributions are updated.

- If each bandit is Bernoulli, then the prior can be modeled as a **Beta distribution**:

  $$p(\theta_k) = \frac{\Gamma(\alpha_k + \beta_k)}{\Gamma(\alpha_k)\Gamma(\beta_k)} \theta_k^{\alpha_k - 1}(1 - \theta_k)^{\beta_k - 1}$$

  where $\Gamma(n) = (n-1)!$. This has mean $\alpha/(\alpha + \beta)$. The larger $\alpha + \beta$ is, the more concentrated the distribution is. Because the beta distribution is the conjugate distribution for the Bernoulli distribution, the posterior is also the beta distribution. The closed form solution to the Bayesian update for $a_t = k$ is $(\alpha_k, \beta_k) := (\alpha_k + r_t, \beta_k + (1 - r_t))$.

- The distinction between a greedy bandit and a Thompson-sampled bandit is that the former has $\overline{\theta}_k := \alpha_k/(\alpha_k + \beta_k)$ whereas the latter samples $\overline{\theta}_k \sim \text{beta}(\alpha_k, \beta_k)$.

Exploration via Posterior Sampling of Q functions: We can extend this to Q functions by representing a **posterior distribution** of Q functions. At each step, we sample from $\overline{Q} \sim P(Q)$ and choose action $a_t = \text{argmax}_a \overline{Q}(s_t, a)$. Then, $P(Q)$ is updated with hte new collected exerience tuples.

- This can be done using **Bayesian neural networks** that estimate the posteriors, neural network ensembles (with or without a shared backbone), or ensembling by dropout (by randomly masking network weights to approximate an ensemble).

9.3 Intrinsic Rewards

Exploration reward bonuses: Intrinsic (task-independent) exploration can be insentivised by augmenting the task-dependent rewards $r(s, a, s')$ with an intrinsic reward:

$$r_{\text{total}} = r_{\text{extrinsic}} + r_{\text{intrinsic}}.$$

Note that the exploration reward bonuses are non-stationary.

Count-based exploration with DeepHashing [36]: In non-tabular environements, complex states such as images can be tabulated in latent compressed state (e.g., using an autoencoder). The intrinsic reward is then inversely proportion to the state visitation count:

$$r_{\text{intrinsic}}(o) \propto 1/\sqrt{N(z)}$$

Model error as an intrinsic reward: Let $T_\theta(s, a)$ denote the learned dynamics function to an embedding space, and $E_\phi(s)$ to be a state-embedding function in the same latent space. Then, we can use an intrinsic reward proportional to

$$r_{\text{intrinsic}}(s, a, s') = \|T_\theta(s, a) - E_\phi(s')\|^2.$$

To address the Noisy-TV problem, we introduce an inverse dynamics model Inv_ψ to predict things that the agent can control:

$$r_{\text{intrinsic}} = \|\text{Inv}_\psi(E_\phi(s), E_\phi(s')) - a\|^2.$$

"Noisy-TV" Problem: The prediction error of the transition function is not necessary a useful signal for the **generalization gap** (high error on novel states unseen during training). Instead, the error may reflect the inherent stochasticity of the environment (aleatoric uncertainty), epistemic uncertainty (agent lacks data) or limitations of the model representation.

Random Network Distillation (RND) [37]: We use a synthetic prediction task for uncertainty estimation. Let $\hat{f} : \mathcal{S} \to \mathbb{R}$ be a randomly initialized neural network. We train an identical neural network initialized independently, and use this to estimate uncertainty. This eliminates transition stochasticity because we model the observation space as a deterministic function of some latent variable (modeled by s') and eliminates mode-mismatch because the predictor shares the same architecture.

Episodic Curiosity through Reachability: Intrinsic rewards are estimated with a non-parametric memory structure populated with embeddings of past image observations. Curiosity reward uses a comparator neural

network that takes as input two images and predicts whether they are temporally close (few actions apart). Specifically, each state is embedded and the neural network is trained with **temporal contrastive learning**:

$$\mathcal{L}_\phi(o, o^+, o^-) = \|E_\phi(o) - E_\phi(o^+)\| + \max(0, \gamma - \|E_\phi(o) - E_\phi(o^-)\|).$$

The agent is then trained on the combined reward using a standard PPO algorithm.

- Specifically, the memory buffer stores an embedding if the curiosity bonus of a new state exceeds a novelty threshold b_{novelty} and has finite capacity K, after which any new state replaces an arbitrarily selected index in the buffer.
- Each state is compared sequentially with every state in the buffer using the comparator neural network and scored according to the 90-th percentile of closest state.

Detachment Problem: In intrinsic-motivation exploration, suppose the environment has multiple promising regions with intrinsic reward. The agent starts exploring one region, consumes the novelty bonus there, and drifts away to another novel region. Once it has "detached" from the first region, the incentive gradient to back to the region has disappeared, so the region remains underexplored.

- Replay buffers should solve this in theory but fail in practice because the buffer size needs to be large, which causes optimization/stability issues.

Derailment problem: Most reinforcement learinng algorithms perturb a promising policy and hope that it explores further. In most cases, however, this breaks the policy, especially as the length, complexity, and precision of the sequence increases.

Go-Explore [38]: Addresses the detachment problem by explicitly tracking an archive of trajectories that reached a particular state in the world. This archive is updated with new states reached, shorter trajectories to reach already archived states, and updates to the **state connectivity map**. This also addresses derailment by returning to prior states using known trajectories and then exploring. A trajectory is only made robust once it has been completely explored and identified as promising.

The second phase is to **robustify** the best trajectory using imitation learning on the best trajectory.

- **Learning Montezuma's Revenge from a Single Demonstration** [39] a similar idea of using a model-free RL method to solve progressively longer suffixes of an expert trajectory to reduce reward sparsity. This is used to robustify an expert demonstration.

10 Application of RL

10.1 Sim2Real Learning

Pros and Cons of Simulation in RL: Simulations are necessary because they allow many samples in a safe, inexpensive environment while avoiding damage to a physical robot. However, simulations suffer from **under-modelling** (hard to replicate all mechanics of the real world), **incorrect parameters** (e.g., friction coefficient, textures, etc.), and systematic discrepencies in dynamics and observations.

Domain Randomization [40, 41]: Simulation environments are diversified by randomizing textures and camera

viewpoints (Figure 2), allowing policies trained in such environments to generalize more effectively.

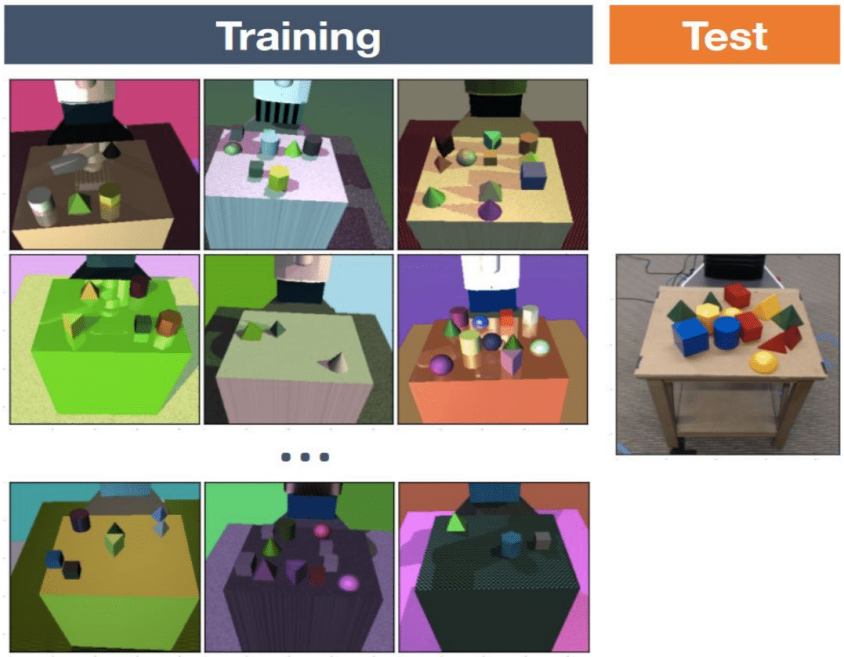


Figure 2: Examples of environments generated with domain randomization

Solving Rubik's Cube with a robot hand **[42]**: The policy was trained entirely in simulation (MuJoCo) using automatic domain randomization, where the cube size, mass, friction coefficient, and visual texture are varied in simulation. The state (position, orientation, face positions) are estimated from camera angles. The overall design is shown in Figure 3.

Automatic Domain Randomization (ADR): Uses a form of curriculum learning where the simulation environment parameters vary more and more from a baseline configuration as the training progresses. Specifically, ADR uses **boundary sampling** to evaluate the policy at boundary of seen paramaters. The distribution is then adjusted (either widened or narrowed) based on the performance of the policy.

Rapid Motor Adaption [43]: Policy learns to walk in simulation on a series of randomized terrains and environment parameters. In training, these parameters are given and encoded into a vector z_t with an encoder. During deployment, these are estimated online using previous state and actions.

Combining Motion Tracking and Adaptibility [44]: Uses a two-phase training pipeline. **AnyTracker** learns a general motion tracking policy in a simulator with no domain randomization (uses specialist-to-generalist training with DAgger). **AnyAdapter** uses history and a world model to update LoRA-style adaptors to adjust actions, without modifying the base tracker.

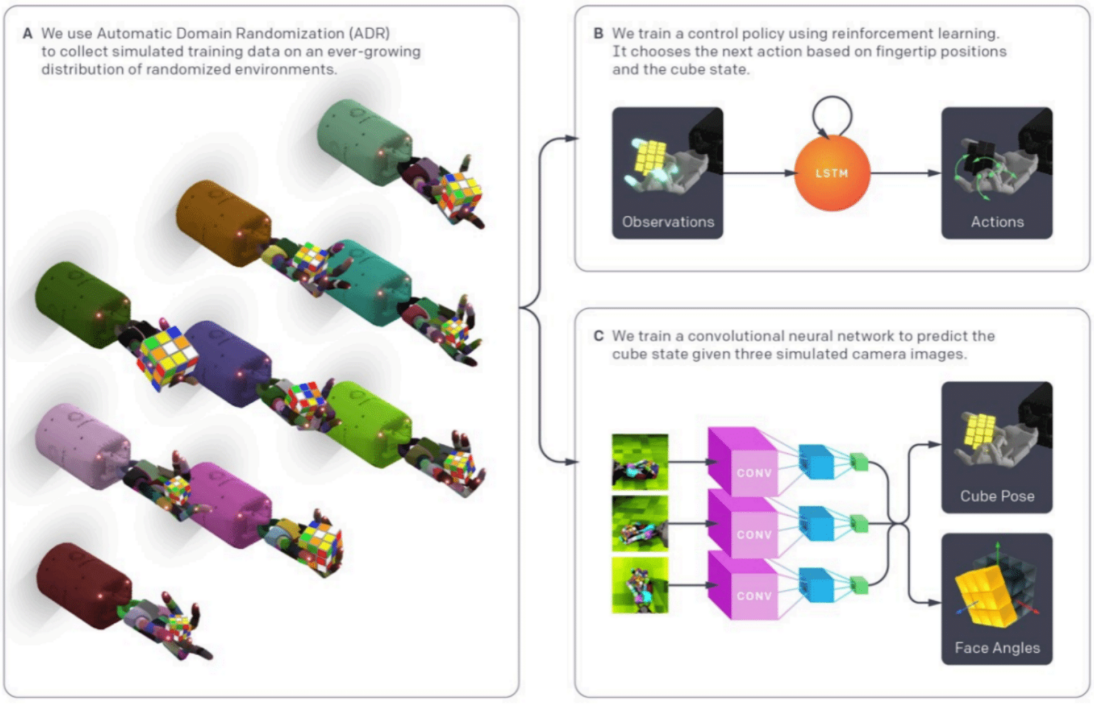


Figure 3: Architecture for RL policy used by *Solving Rubik's Cube with a robot hand*

10.2 Foundation Models for RL

Extraction Action Plans from Language Models/Inner Monologue [45, 46]: One-shot/few-shot prompting can be used to decompose a task into several subtasks. Using pre-selected action decompositions, this allows an agent to complete complex tasks. This can be augemnted with a visual language model to dynamically adjust plans (e.g., if a certain subtask fails).

Code as Policies [47]: Use language models to write Python code that composes API calls to assemble new robot behaviors for tasks. The API can be provided via examples in the prompt or using **retrieval-augmented generation (RAG)**.

In-Context Abstraction Learning (ICAL) [48]: Convertsr plans into programs of thought with human-in-the-loop and environment feedback. For each task that the agent attempts, humans provide feedback in natural language. The agent then rewrites the prompts with the new, updated task decomposition. This process is used to generate demonstrations for future tasks.

- Because the memory is effectively an "expert", a model provided with its own past demonstrations perform better than one seeded with human-designed demonstrations.
- This can applied to tasks like visual web navigation, in addition to physical environment manipulation.

Using LLMs to generate Reward Functions: Reward functions for complex tasks can be encoded via language models, which enables standard RL algorithms such as PPO to learn complex tasks.

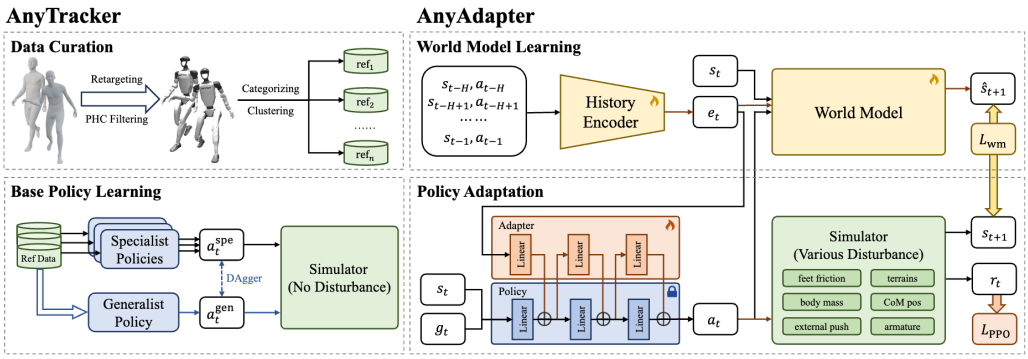


Fig. 2: **Overview of our method. Any2Track** consists of two key components: AnyTracker and AnyAdapter. **AnyTracker** is a general motion tracker with a series of careful designs. **AnyAdapter** is a history-informed adaptation module on top of AnyTracker. AnyAdapter endows the base tracker with online dynamics adaptability without compromising its fundamental expressive motion tracking capability.

Figure 4: Two-phase RL training process for *AnyTracker/AnyAdapter*

Eureka [49]: Rewards for a task are iteratively refined by a language model based on task performance in a simulation environment.

CLIP as a Reward Model [50]: A visual foundation model such as CLIP can be used as the reward function for an RL model. The VLM takes in two trajectories performed by the policy and is asked to rank them; this is then used as the preference label for the RL policy.

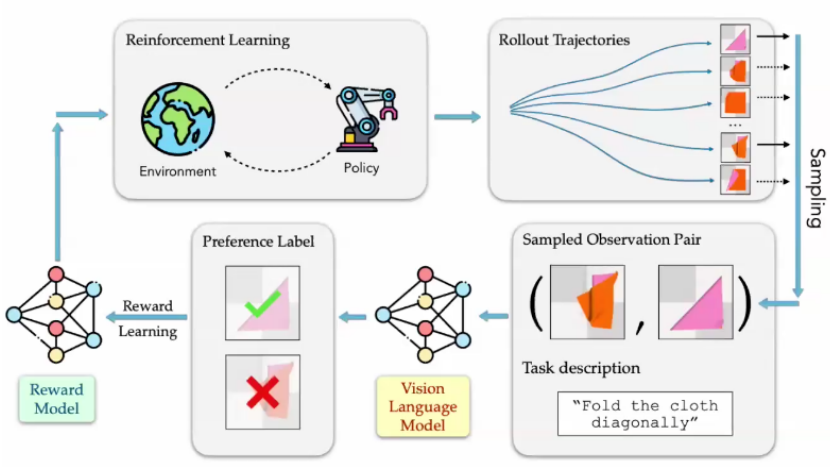


Figure 5: Schematic of using visual language models for rewards for training a policy

Dex4D [51]: Introduces the **Anypose-to-Anypose (AP2AP)** that abstracts dexterous manipulation as transforming an object from an abitrary initial 3D pose to an arbitrary target 3D pose, trained on 3,200 UniDexGrasp objects with diverse poses and domain randomization. Training uses a **paired point encoding** (encodes current and target positions, preserving point correspondence) and the teacher-student framework, where a privileged teacher policy is trained with PPO in simulation, and then a student policy is stilled with DAgger with partial observability (Figure 6). At test time, Dex4D uses a visual generation model to produce a future RDB video,

starting from a language instruction an initial RGBD observation (depth generated separately using **Video Depth Anything**).

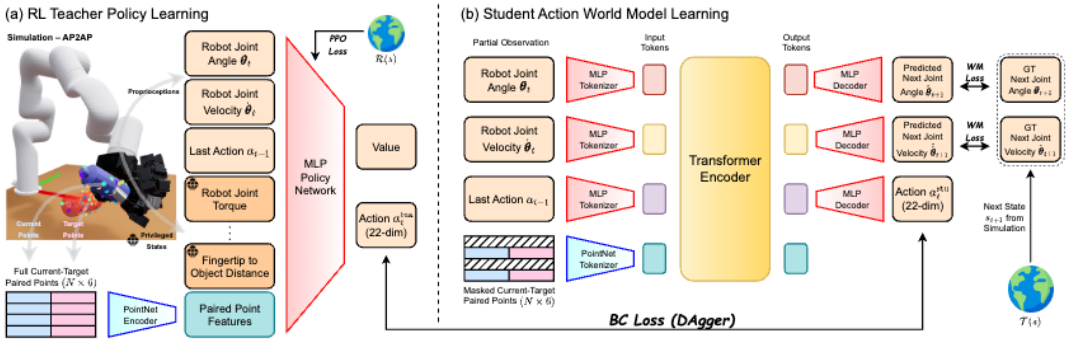


Fig. 2: Overview of our Dex4D teacher and student network architectures. **(a)** We first learn a teacher policy via RL with privileged states and full points sampled on the whole object, leveraging our proposed **Paired Point Encoding** representation. **(b)** Given partial observation, *i.e.,* robot proprioception, last action, and masked paired points, we distill from the teacher and learn a **transformer-based student action world model** that jointly predicts actions and future robot states.

Figure 6: Dex4D teacher-student training framework

10.3 RL for Foundation Models

Text Generation as an MDP: For a prompt x and token $y_1, \ldots, y_t$, we represent state $s_t = (x, y_{<t})$, action $a_t = y_t$, and policy $\pi_t heta(a_t \mid s_t) = \pi_\theta(y_t \mid x, y_{<t})$.

Reinforcement Learning From Human Feedback (RLHF) [52]: We first train a reward model using supervised learning using explemary human demonstrations (learning tone, format, etc.). Specifically, the feedback comes as preferences over model samples $\mathcal{D} = \left\{x^{(i)}, y_w^{(i)}, y_\ell^{(i)}\right\}$, corresponding to the prompt, preferred response, and dispreferred response, respectively. The **Bradley-Terry model** connects rewards to preferencs via

$$p(y_w \succ y_\ell \mid x) = \sigma(r(x, y_w) - r(x, y_\ell)).$$

Thus, the reward model is trained via

$$\mathcal{L}_{\mathcal{D}}(\phi) = -\mathbf{E}_{(x,y_w,y_\ell)\sim\mathcal{D}}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_\ell))\right]$$

In practice, this is implemented as an LLM. Then, a pretrained LLM $\pi_{\text{pretrained}}$ is fine-tuned using SFT on a curated expert response dataset to π_{SFT}. Finally, the reward model r_ϕ is frozen and used to learn a policy π_θ via

$$\max_\theta \mathbf{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(y|x)}\left[r_\phi(x, y)\right] - \beta\mathcal{D}_{\text{KL}}\left(\pi_\theta(y \mid x) \| \pi_{\text{ref}}(y \mid x)\right)$$

This is implemented as a TRPO-style or PPO-style loss:

$$\max_\theta \hat{\mathbf{E}} \sum_{t=1}^{T} \left(\frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{SFT}}(y_t \mid x, y_{<t})} \hat{A}_t - \beta\mathcal{D}_{\text{KL}}\left(\pi_\theta(y \mid x) \| \pi_{\text{ref}}(y \mid x)\right) \right)$$

The overall objective is visualized in Figure 7.

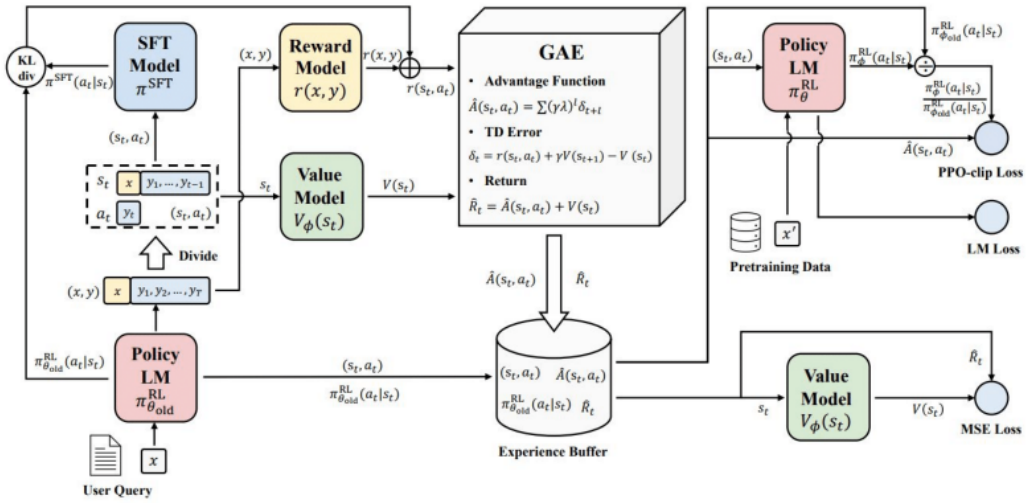


Figure 7: Overall RLHF formulation

Generalized Advantage Estimation (GAE): An advantage estimation method that is a weighted average of TD learning and Monte-Carlo. Specifically,

$$\hat{A}_t^{\mathsf{GAE}(\gamma,\delta)} = \sum_{i=0}^{\infty} (\gamma\delta)^i \delta_{t+i}$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the TD error. In particular, $\gamma = 0$ gives TD(0) (low variance, high bias) and $gamma = 1$ gives Monte Carlo (low bias, high variance).

Direct Preference Optimization (DPO) [53]: The closed-form optimal policy of the RLHF objective is given by

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\mathsf{SFT}}(y \mid x) \exp\left(\frac{1}{\beta} r(x,y)\right),$$

where $Z(x) = \sum_y \pi_{\mathsf{SFT}}(y \mid x) \exp\left(\frac{1}{\beta} r(x,y)\right)$. See Advantage-Weighted Regression and Boltzmann Reweighting for details on this derivation. Solving the RLHF objective for the reward function yields

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathsf{SFT}}(y \mid x)} + \beta \log Z(x).$$

In particular, the ration $\pi^*(y \mid x)/\pi_{\mathsf{SFT}}(y \mid x)$ is positive if the policy likes a response more than the reference model. Then, substituting into Bradley-Terry model-based loss function, we obtain the DPO loss:

$$\mathcal{L}_{\mathsf{DPO}}(\pi_\theta; \pi_{\mathsf{SFT}}) = -\mathbf{E}_{(x,y_w,y_\ell)\sim\mathcal{D}} \log \sigma \left(\underbrace{\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathsf{SFT}}(y_w \mid x)}}_{\text{reward of preferred response}} - \underbrace{\beta \log \frac{\pi_\theta(y_\ell \mid x)}{\pi_{\mathsf{SFT}}(y_\ell \mid x)}}_{\text{reward of dispreferred response}} \right).$$

Thus, the implicit reward here is

$$\hat{r}_\theta(x,y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\mathsf{SFT}}(y \mid x)}.$$

The gradient of the loss is given by

$$\nabla_\theta \mathcal{L}_{\mathsf{DPO}}(\pi_\theta; \pi_{\mathsf{SFT}}) = -\beta \mathbf{E}_{(x,y_w,y_\ell)\sim\mathcal{D}} \sigma(\hat{r}_\theta(x, y_\ell) - \hat{r}_\theta(x, y_w)) \left(\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_\ell \mid x)\right).$$

Group Relative Policy Optimization [54, 55]: For a given prompt x, the policy model samples several outputs $y_1, \ldots, y_G$, each of which is scored by a reward model yielding $r_1, \ldots, r_G$. The advantage score of each answer is estimated relative to the group:

$$A_i = \frac{r_i - \mathsf{mean}(r)}{\mathsf{std}(r)}.$$

This allows GRPO to avoid learning the value model (used by GAE), as shown in Figure 8. The overall loss follows a PPO-style objective:

$$\mathcal{L}_{\mathsf{GRPO}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \min(r_\theta(\tau)A(\tau), \mathsf{clip}(r_\theta(\tau), 1-\epsilon, 1+\epsilon)A(\tau)) - \beta \mathcal{D}_{\mathsf{KL}}(\pi_\theta(\cdot \mid x) \| \pi_{\mathsf{SFT}}(\cdot \mid x)).$$

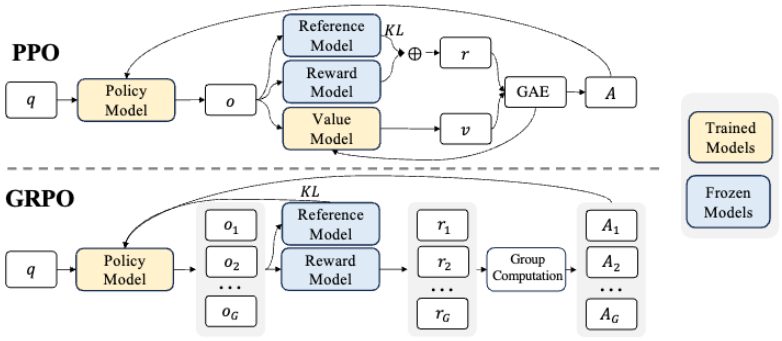


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

Figure 8: Comparison of computations done by GRPO vs PPO

Chain-of-Thought Reasoning with RL in VLMs [56, 57, 58]: Chain-of-thought reasoning in visual models in pure text-space is ineffective (¡2 visual regions examined, little-to-no visually-grounded subgoal setting, verification, or backtracking). Grounded CoT associates thoughts produced by the language model with a location within the image of interest.

Grounded RL for Visual Reasoning [59]: The training data is warm-started using MCTS, using a strong teachem model (Qwen2.5-VL-72B) that samples grounded reasoning steps, keeping successful trajectories. Then, a smaller base VLM is fine-tuned using supervised learning on the curated trajectories. Finally, GRPO with task correctness as a reward is used to fine tune the SFT model. See Figure 9.

10.4 Training Diffusion Models with RL

Diffusion Models with RL: Consider we have a reward function over generated samples $\mathbf{x}_0$ and contexts $\mathbf{c}$. We are trying to optimize

$$J_{\mathsf{DDRL}}(\theta) = \mathbf{E}_{\mathbf{c}\sim p(\mathbf{c}), \mathbf{x}_0 \sim p_\theta(\mathbf{x}_0|c)} r(\mathbf{x}_0, \mathbf{c}),$$

where r may or may not be differentiable w.r.t. $\mathbf{x}_0$.

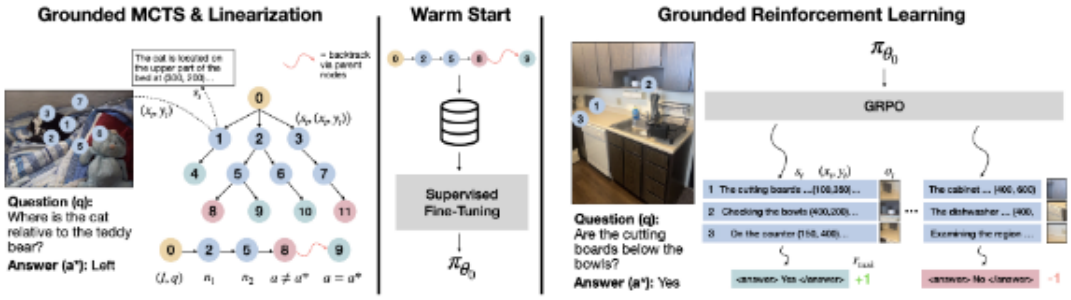


Figure 3: **Overview of the ViGoRL approach.** (Left) We use MCTS with a teacher model to generate reasoning chains grounded in specific image regions. (Middle) These reasoning trees are linearized and used for supervised fine-tuning (SFT) to train a base model. (Right) We apply GRPO with an outcome-based reward to further refine the grounded reasoning.

Figure 9: Overview of the ViGoRL algorithm

- Examples of rewards include scoring aesthetics, text-alignment of generated images, or scoring trajectories obtained by an aciton diffusion policy.

Reward-weighted Regression (RWR): RWR trains a policy to imitate high-return episodes more strongly than low-return episodes. Specifically, given a parametric policy $\pi_\theta(a \mid s)$, a dataset of episodes τ_i, and the returns of each episode R_i, RWR solves

$$\theta^* = \operatorname*{argmax}_\theta \sum_{i=1}^{N} \sum_{t=1}^{T_i} w_i \log \pi_\theta(a_{i,t} \mid s_{i,t}),$$

where $w_i = \exp(\beta R_i) / \sum_j \exp(\beta R_j)$. This is similar to offline RL, except in diffusion models we do not have access to $\pi_\theta(a_{i,t} \mid s_{i,t})$. The specific weight function can take many forms:

$$w_{\text{RWR}}(\mathbf{x}_0, \mathbf{c}) = \frac{1}{Z} \exp(\beta r(\mathbf{x}_0, \mathbf{c}))$$
$$w_{\text{sparse}}(\mathbf{x}_0, \mathbf{c}) = \mathbf{1}[r(\mathbf{x}_0, \mathbf{c}) \geq C]$$

The latter is similar to filtered behavior cloning.

Reward-Weighted Diffusion [60]: This combined RWR with diffusion-denoising. The weighted diffusion loss is given by:

$$\mathcal{L}_{\text{RWR}}(\theta) = \mathbf{E}_{(\mathbf{x}_0,\mathbf{c})\sim p(\mathbf{x}_0,c), t\sim\mathcal{U}[0,T], \mathbf{x}_t\sim q(\mathbf{x}_t|\mathbf{x}_0)} \left[w(\mathbf{x}_0, c) \|\tilde{\boldsymbol{\mu}}(\mathbf{x}_0, t) - \boldsymbol{\mu}_\theta(\mathbf{x}_0, t)\|^2 \right].$$

To approximate this, we collect human feedback and train a reward model via

$$\mathcal{L}^{\text{MSE}}(\phi) = \mathbf{E}_{(\mathbf{x}_0,\mathbf{c},y)\sim\mathcal{D}_{\text{human}}} (y - r_\phi(\mathbf{x}_0, c))^2.$$

Denoising as a multi-step MDP: We treat the process of denoising as a MDP where the state $s_t = (\mathbf{c}, t, \mathbf{x}_t)$, the action $a_t = \mathbf{x}_{t-1}$, and the policy $\pi(a_t \mid s_t) = p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t, c)$ (a Gaussian with fixed variance and parameterized mean). The initial state distribution $\rho_0(s_0) = (p(\mathbf{c}), \delta_T, \mathcal{N}(\mathbf{0}, \mathbf{I}))$ and the dynamics are given by $P(s_{t+1} \mid s_t, a_t) = (\delta_{\mathbf{c}}, \delta_{t-1}, \delta_{\mathbf{x}_{t-1}})$. Finally, the rewards are given by $R(s_t, a_t) = r(\mathbf{x}_0, \mathbf{c})$ for $t = 0$, and 0

otherwise. This allows policy-gradients to apply as on standard Gaussian policies:

$$\nabla_\theta J_{\text{DDRL}} = \mathbf{E} \sum_{t=0}^{T} \nabla_\theta \log p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t, c) r(\mathbf{x}_0, c)$$

In particular, the $\log p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t, c)$ is a Gaussian policy. Recall that the gradient of a Gaussian policy is given by

$$\nabla_\theta \log \pi_\theta(a \mid s) = C \cdot \frac{(a - \mu_\theta(s))}{\sigma^2} \nabla_\theta \mu_\theta(s)$$

where the variance σ^2 is fixed and C is some constant.

Off-Policy and PPO-style Denoising Losses: Suppose we sample instead from a frozen model p_{old}. Define $\rho_t(\theta) = \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{c})}{p_{\text{old}}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{c})}$. Then, the off-policy loss is given by

$$\nabla_\theta J_{\text{DDRL-IS}} = \mathbf{E}_{\mathbf{x}_{T,\ldots,0} \sim p_{\text{old}}} \sum_{t=0}^{T} \rho_t(\theta) \nabla_\theta \log p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t, c) r(\mathbf{x}_0, c)$$

With PPO-style trust region constraints, this becomes

$$J_{\text{PPO-CLIP}} = \mathbf{E}_{\mathbf{x}_{T,\ldots,0} \sim p_{\text{old}}} \sum_{t=0}^{T} \min(\rho_t(\theta) r(\mathbf{x}_0, \mathbf{c}), \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon) r(\mathbf{x}_0, \mathbf{c})$$

$$\nabla_\theta J_{\text{PPO-CLIP}} = \mathbf{E}_{\mathbf{x}_{T,\ldots,0} \sim p_{\text{old}}} \sum_{t=0}^{T} w_t^{\text{clip}}(\theta) \nabla_\theta \log p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t, c) A_t$$

Diffusion Policy Policy Optimization (DPPO) [61]: DPPO is a fine-tunining technique that operates on robot diffusion policies using PPO-style policy gradients, as opposed to treating the diffusion policy only as a behavior-cloned generator. Actions are denoised by a diffusion policy via $a_t^{(K)} \to a_t^{(K-1)} \to \cdots \to a_t^{(0)}$, and then $a_t = a_t^{(0)}$ is executed in the environment yielding a reward r_t and transitioning to s_{t+1}. This frames an RL problem as a two-nested MDP:

$$\begin{aligned}
\overline{r}(\overline{s}_t, \overline{a}_t) &= \sum_{t' \geq t} \gamma_{\text{env}}^t \overline{r}(\overline{s}_{\overline{t}}(t', 0), \overline{a}_{\overline{t}}(t', 0)) \\
\hat{A}^{\pi_{\text{old}}} &= \gamma_{\text{denoise}}^k \left(\overline{r}(\overline{s}_t, \overline{a}_t) - \hat{V}^{\pi_{\text{old}}}(\overline{s}_t(t, 0)) \right) \\
\hat{V}^{\pi_{\text{old}}}(\overline{s}_t(t, 0)) &= \tilde{V}^{\pi_{\text{old}}}(s_t) \\
\nabla_\theta \mathbf{E}_{(s_t, a_t) \sim \pi_{\text{old}}} \min &\left(\hat{A}^{\pi_{\text{old}}}(s_t, a_t) \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}, \hat{A}^{\pi_{\text{old}}}(s_t, a_t) \text{clip} \left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon \right) \right)
\end{aligned}$$

where $\overline{t} = \overline{t}(t, k)$. See Figure 10.

AlignProp [62, 63]: Fine-tuning method for diffusion models that backpropogates the reward gradient through the diffusion sampling process.

$$\begin{aligned}
\mathcal{L}(\theta) &= -\mathbf{E}_{\mathbf{c}, \mathbf{x}_{T,\ldots,0} \sim p_\theta} R(\mathbf{x}_0, \mathbf{c}) \\
\nabla_\theta \mathcal{L}(\theta) &= -\frac{\partial R}{\partial \mathbf{x}_0} \frac{\partial \mathbf{x}_0}{\partial \theta},
\end{aligned}$$

Algorithm 1 DPPO

Pre-train diffusion policy π_θ with offline dataset $\mathcal{D}_{\text{off}}$ using BC loss $\mathcal{L}_{\text{BC}}(\theta)$ Eq. (A1).
Initialize value function V_ϕ.
for iteration = 1, 2, … **do**
  Initialize rollout buffer $\mathcal{D}_{\text{itr}}$.
  $\pi_{\theta_{\text{old}}} = \pi_\theta$.
  for environment = 1, 2, …, N *in parallel* **do**
    Initialize state $\bar{s}_{\bar{t}(0,K)} = (s_0, a_0^K)$ in $\mathcal{M}_{\text{DP}}$.
    for environment step t = 1, …, T, denoising step $k = K-1, \dots, 0$ **do**
      Sample the next denoised action $\bar{a}_{\bar{t}(t,k)} = a_t^k \sim \pi_{\theta_{\text{old}}}$.
      if $k = 0$ **then**
        Run a_t^0 in the environment and observe $\bar{R}_{\bar{t}(t,0)}$ and $\bar{s}_{\bar{t}(t+1,K)}$.
      else
        Set $\bar{R}_{\bar{t}(t,k)} = 0$ and $\bar{s}_{\bar{t}(t,k-1)} = (s_t, a_t^k)$.
      Add $(k, \bar{s}_{\bar{t}(t,k)}, \bar{a}_{\bar{t}(t,k)}, \bar{R}_{\bar{t}(t,k)})$ to $\mathcal{D}_{\text{itr}}$.
  Compute advantage estimates $A^{\pi_{\theta_{\text{old}}}}(s_{\bar{t}(t,k=0)}, a_{\bar{t}(t,k=0)})$ for $\mathcal{D}_{\text{itr}}$ using GAE Eq. (A2).
  for update = 1, 2, …, num_update **do** ▷ Based on replay ratio N_θ
    for minibatch = 1, 2, …, B **do**
      Sample $(k, \bar{s}_{\bar{t}(t,k)}, \bar{a}_{\bar{t}(t,k)}, \bar{R}_{\bar{t}(t,k)})$ and $A^{\pi_{\theta_{\text{old}}}}(s_{\bar{t}(t,k)}, a_{\bar{t}(t,k)})$ from $\mathcal{D}_{\text{itr}}$.
      Compute denoising-discounted advantage $\hat{A}_{\bar{t}(t,k)} = \gamma_{\text{DENOISE}}^k A^{\pi_{\theta_{\text{old}}}}(s_{\bar{t}(t,0)}, a_{\bar{t}(t,0)})$.
      Optimize π_θ using policy gradient loss $\mathcal{L}_\theta$ Eq. (A3).
      Optimize V_ϕ using value loss $\mathcal{L}_\phi$ Eq. (A4).
return converged policy π_θ.

Figure 10: DPPO Algorithm

where the second term is propogated through the entire denoising trajectory (similar to BPTT for an RNN).

Diffusion-QL [64]: Offline RL algorithm that learns actions that stay close to the offline dataset but prioritize ones with high learned Q-value. The loss is given by

$$\mathcal{L}_{\text{actor}}(\theta) = \mathcal{L}_{\text{diffusion}}(\theta) - \alpha \mathbf{E}_{s\sim\mathcal{D}, a^{(0)}\sim\pi_\theta(\cdot|s)} Q_\phi(s, a^{(0)})$$

where the first term is the diffusion BC loss and the second term is the Q-learning improvement term. The critic is trained like a standard actor critic method to minimize the squared difference between $y = r + \gamma Q_{\phi'}(s', a'^{(0)})$ and $Q_\phi(s, a)$, where $a'^{(0)} \sim \pi_{\theta'}(\cdot \mid s')$.

Diffusion Steering via Reinforcement Learning (DSRL) [65]: Given a frozen behavior-cloned diffusion policy π_{dp}^w, we train a new policy $\pi^{\mathcal{W}}(w \mid s)$ that operates on the noise-space of π_{dp}. Actions are generated via $\pi_{\text{dp}}(\pi^{\mathcal{W}}(\cdot \mid s))$. DSRL introduces two critics: $Q^{\mathcal{A}}(s, a)$ is trained on action-space transitions (s, a, r, s'); $Q^{\mathcal{W}}(s, w)$ is trained with distillation to approximate

$$Q^{\mathcal{W}}(s, w) \approx Q^{\mathcal{A}}(s, \pi_{\text{dp}}(s, w)).$$

The learned noise actor is optimized against $Q^{\mathcal{W}}$. See Figure 11.

Algorithm 1 Noise-Aliased Diffusion Steering via Reinforcement Learning (DSRL-NA)

input: pretrained diffusion policy $\pi_{\text{dp}}^{\mathcal{W}}$, offline data $\mathcal{D}_{\text{off}}$ and/or online environment $\mathcal{M}$
Initialize replay buffer $\mathfrak{B} \leftarrow \mathcal{D}_{\text{off}}$, $\mathcal{A}$-critic $Q^{\mathcal{A}}$, latent-noise critic $Q^{\mathcal{W}}$, latent-noise actor $\pi^{\mathcal{W}}$
for $t = 1, \dots, T$ **do**
  Update $Q^{\mathcal{A}}$: $\min_{Q^{\mathcal{A}}} \mathbb{E}_{(\boldsymbol{s},\boldsymbol{a},r,\boldsymbol{s}')\sim\mathfrak{B}, \boldsymbol{a}'\sim\pi_{\text{dp}}^{\mathcal{W}}(\boldsymbol{s}', \pi^{\mathcal{W}}(\boldsymbol{s}'))} \left[(Q^{\mathcal{A}}(\boldsymbol{s}, \boldsymbol{a}) - r - \gamma \bar{Q}^{\mathcal{A}}(\boldsymbol{s}', \boldsymbol{a}'))^2\right]$
  Update $Q^{\mathcal{W}}$: $\min_{Q^{\mathcal{W}}} \mathbb{E}_{\boldsymbol{s}\sim\mathfrak{B}, \boldsymbol{w}\sim\mathcal{N}(0,I)} [(Q^{\mathcal{W}}(\boldsymbol{s}, \boldsymbol{w}) - Q^{\mathcal{A}}(\boldsymbol{s}, \pi_{\text{dp}}^{\mathcal{W}}(\boldsymbol{s}, \boldsymbol{w})))^2]$
  Update $\pi^{\mathcal{W}}$: $\max_{\pi^{\mathcal{W}}} \mathbb{E}_{\boldsymbol{s}\sim\mathfrak{B}} \left[Q^{\mathcal{W}}(\boldsymbol{s}, \pi^{\mathcal{W}}(\boldsymbol{s}))\right]$
  if access to online environment $\mathcal{M}$ **then**
    Sample latent-noise action $\boldsymbol{w}_t \sim \pi^{\mathcal{W}}(\boldsymbol{s}_t)$ and compute $\boldsymbol{a}_t \leftarrow \pi_{\text{dp}}^{\mathcal{W}}(\boldsymbol{s}_t, \boldsymbol{w}_t)$
    Play $\boldsymbol{a}_t$ in $\mathcal{M}$, observe r_t and next state $\boldsymbol{s}_{t+1}$, and add $(\boldsymbol{s}_t, \boldsymbol{a}_t, r_t, \boldsymbol{s}_{t+1})$ to $\mathfrak{B}$

Figure 11: DSRL Algorithm

References

[1] Christopher J C H Watkins and Peter Dayan. "Q-learning". In: *Machine Learning* 8.3–4 (1992), pp. 279–292.

[2] Volodymyr Mnih et al. "Human-level control through deep reinforcement learning". In: *Nature* 518.7540 (2015), pp. 529–533.

[3] Hado van Hasselt, Arthur Guez, and David Silver. "Deep Reinforcement Learning with Double Q-learning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30. 1. 2016.

[4] Scott Fujimoto, Herke van Hoof, and David Meger. "Addressing Function Approximation Error in Actor-Critic Methods". In: *Proceedings of the 35th International Conference on Machine Learning*. PMLR. 2018, pp. 1587–1596.

[5] Tom Schaul et al. "Prioritized Experience Replay". In: *International Conference on Learning Representations*. 2016.

[6] Ronald J Williams. "Simple statistical gradient-following algorithms for connectionist reinforcement learning". In: *Machine learning* 8.3 (1992), pp. 229–256.

[7] Richard S Sutton et al. "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In: *Advances in Neural Information Processing Systems*. Vol. 12. MIT Press, 1999, pp. 1057–1063.

[8] Volodymyr Mnih et al. "Asynchronous Methods for Deep Reinforcement Learning". In: *International Conference on Machine Learning*. PMLR. 2016, pp. 1928–1937.

[9] John Schulman et al. "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).

[10] John Schulman et al. "Trust Region Policy Optimization". In: *International Conference on Machine Learning*. PMLR. 2015, pp. 1889–1897.

[11] Timothy P Lillicrap et al. "Continuous control with deep reinforcement learning". In: *arXiv preprint arXiv:1509.02971* (2016).

[12] Tuomas Haarnoja et al. "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor". In: *International Conference on Machine Learning*. PMLR. 2018, pp. 1861–1870.

[13] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning". In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. PMLR. 2011, pp. 627–635.

[14] Ian Goodfellow et al. "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014.

[15] Jonathan Ho and Stefano Ermon. "Generative Adversarial Imitation Learning". In: *Advances in Neural Information Processing Systems*. Vol. 29. 2016.

[16] Marcin Andrychowicz et al. "Hindsight Experience Replay". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[17] Diederik P Kingma and Max Welling. "Auto-Encoding Variational Bayes". In: *International Conference on Learning Representations*. 2014.

[18] Jonathan Ho, Ajay Jain, and Pieter Abbeel. "Denoising Diffusion Probabilistic Models". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 6840–6851.

[19] Xiaoxiao Guo et al. "Deep learning for real-time Atari game play using offline Monte-Carlo tree search planning". In: *Advances in neural information processing systems* 27 (2014).

[20] David Silver et al. "Mastering the game of Go with deep neural networks and tree search". In: *nature* 529.7587 (2016), pp. 484–489.

[21] David Silver et al. "Mastering chess and shogi by self-play with a general reinforcement learning algorithm". In: *arXiv preprint arXiv:1712.01815* (2017).

[22] Jessica B Hamrick et al. "On the role of planning in model-based deep reinforcement learning". In: *arXiv preprint arXiv:2011.04021* (2020).

[23] Kurtland Chua et al. "Deep reinforcement learning in a handful of trials using probabilistic dynamics models". In: *Advances in neural information processing systems* 31 (2018).

[24] Junhyuk Oh et al. "Action-conditional video prediction using deep networks in atari games". In: *Advances in neural information processing systems* 28 (2015).

[25] Lukasz Kaiser et al. "Model-based reinforcement learning for atari". In: *arXiv preprint arXiv:1903.00374* (2019).

[26] Nicklas Hansen, Xiaolong Wang, and Hao Su. "Temporal difference learning for model predictive control". In: *arXiv preprint arXiv:2203.04955* (2022).

[27] Julian Schrittwieser et al. "Mastering atari, go, chess and shogi by planning with a learned model". In: *Nature* 588.7839 (2020), pp. 604–609.

[28] Julian Schrittwieser et al. "Online and offline reinforcement learning by planning with a learned model". In: *Advances in Neural Information Processing Systems* 34 (2021), pp. 27580–27591.

[29] Michael Janner et al. "Planning with diffusion for flexible behavior synthesis". In: *arXiv preprint arXiv:2205.09991* (2022).

[30] Brian Yang et al. *Diffusion-ES: Gradient-free Planning with Diffusion for Autonomous Driving and Zero-Shot Instruction Following*. 2024. arXiv: 2402.06559 [cs.LG].

[31] Scott Fujimoto and Shixiang Shane Gu. "A minimalist approach to offline reinforcement learning". In: *Advances in neural information processing systems* 34 (2021), pp. 20132–20145.

[32] Scott Fujimoto, David Meger, and Doina Precup. "Off-policy deep reinforcement learning without exploration". In: *International conference on machine learning*. PMLR. 2019, pp. 2052–2062.

[33] Aviral Kumar et al. "Conservative Q-Learning for Offline Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1179–1191.

[34] Xue Bin Peng et al. "Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning". In: *arXiv preprint arXiv:1910.00177* (2019).

[35] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. "Offline Reinforcement Learning with Implicit Q-Learning". In: *International Conference on Learning Representations*. 2022.

[36] Haoran Tang et al. "# exploration: A study of count-based exploration for deep reinforcement learning". In: *Advances in neural information processing systems* 30 (2017).

[37] Yuri Burda et al. "Exploration by Random Network Distillation". In: *International Conference on Learning Representations*. 2019.

[38] Adrien Ecoffet et al. "First return, then explore". In: *Nature* 590.7847 (2021), pp. 580–586.

[39] Tim Salimans and Richard Chen. "Learning montezuma's revenge from a single demonstration". In: *arXiv preprint arXiv:1812.03381* (2018).

[40] Josh Tobin et al. "Domain randomization for transferring deep neural networks from simulation to the real world". In: *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE. 2017, pp. 23–30.

[41] Jonathan Tremblay et al. "Training deep networks with synthetic data: Bridging the reality gap by domain randomization". In: *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*. 2018, pp. 969–977.

[42] Ilge Akkaya et al. "Solving rubik's cube with a robot hand". In: *arXiv preprint arXiv:1910.07113* (2019).

[43] Ashish Kumar et al. "Rma: Rapid motor adaptation for legged robots". In: *arXiv preprint arXiv:2107.04034* (2021).

[44] Zhikai Zhang et al. "Track any motions under any disturbances". In: *arXiv preprint arXiv:2509.13833* (2025).

[45] Wenlong Huang et al. "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents". In: *International conference on machine learning*. PMLR. 2022, pp. 9118–9147.

[46] Wenlong Huang et al. "Inner monologue: Embodied reasoning through planning with language models". In: *arXiv preprint arXiv:2207.05608* (2022).

[47] Jacky Liang et al. "Code as policies: Language model programs for embodied control". In: *2023 IEEE International conference on robotics and automation (ICRA)*. IEEE. 2023, pp. 9493–9500.

[48] Gabriel Sarch et al. "Open-ended instructable embodied agents with memory-augmented large language models". In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023, pp. 3468–3500.

[49] Yecheng Jason Ma et al. "Eureka: Human-level reward design via coding large language models". In: *arXiv preprint arXiv:2310.12931* (2023).

[50] Juan Rocamonde et al. "Vision-language models are zero-shot reward models for reinforcement learning". In: *arXiv preprint arXiv:2310.12921* (2023).

[51] Yuxuan Kuang et al. "Dex4D: Task-Agnostic Point Track Policy for Sim-to-Real Dexterous Manipulation". In: *arXiv preprint arXiv:2602.15828* (2026).

[52] Long Ouyang et al. "Training language models to follow instructions with human feedback". In: *Advances in neural information processing systems* 35 (2022), pp. 27730–27744.

[53] Rafael Rafailov et al. "Direct preference optimization: Your language model is secretly a reward model". In: *Advances in neural information processing systems* 36 (2023), pp. 53728–53741.

[54] Daya Guo et al. "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning". In: *Nature* 645.8081 (2025), pp. 633–638.

[55] Zhihong Shao et al. "Deepseekmath: Pushing the limits of mathematical reasoning in open language models". In: *arXiv preprint arXiv:2402.03300* (2024).

[56] Ziyu Liu et al. "Visual-rft: Visual reinforcement fine-tuning". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2025, pp. 2034–2044.

[57] Zhengxi Lu et al. "Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 21. 2026, pp. 17608–17616.

[58] Hao Shao et al. "Visual cot: Advancing multi-modal language models with a comprehensive dataset and benchmark for chain-of-thought reasoning". In: *Advances in Neural Information Processing Systems* 37 (2024), pp. 8612–8642.

[59] Gabriel Sarch et al. "Grounded reinforcement learning for visual reasoning". In: *arXiv preprint arXiv:2505.23678* (2025).

[60] Kimin Lee et al. "Aligning text-to-image models using human feedback". In: *arXiv preprint arXiv:2302.12192* (2023).

[61] Allen Z Ren et al. "Diffusion policy policy optimization". In: *arXiv preprint arXiv:2409.00588* (2024).

[62] Mihir Prabhudesai et al. “Aligning text-to-image diffusion models with reward backpropagation”. In: (2023).

[63] Mihir Prabhudesai et al. “Video diffusion alignment via reward gradients”. In: *arXiv preprint arXiv:2407.08737* (2024).

[64] Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. “Diffusion policies as an expressive policy class for offline reinforcement learning”. In: *arXiv preprint arXiv:2208.06193* (2022).

[65] Andrew Wagenmaker et al. “Steering your diffusion policy with latent space reinforcement learning”. In: *arXiv preprint arXiv:2506.15799* (2025).